DuocUC® INFORMÁTICA Y TELECOMUNICACIONES

Calidad de Software

FORMAMOS **PERONAS** PARA UNA **SOCIEDAD** MEJOR

ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

Recordemos sobre Gestión de Proyectos de Ágil Software

DuocUC®

03

Planificación Ágil

ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

KANBAN

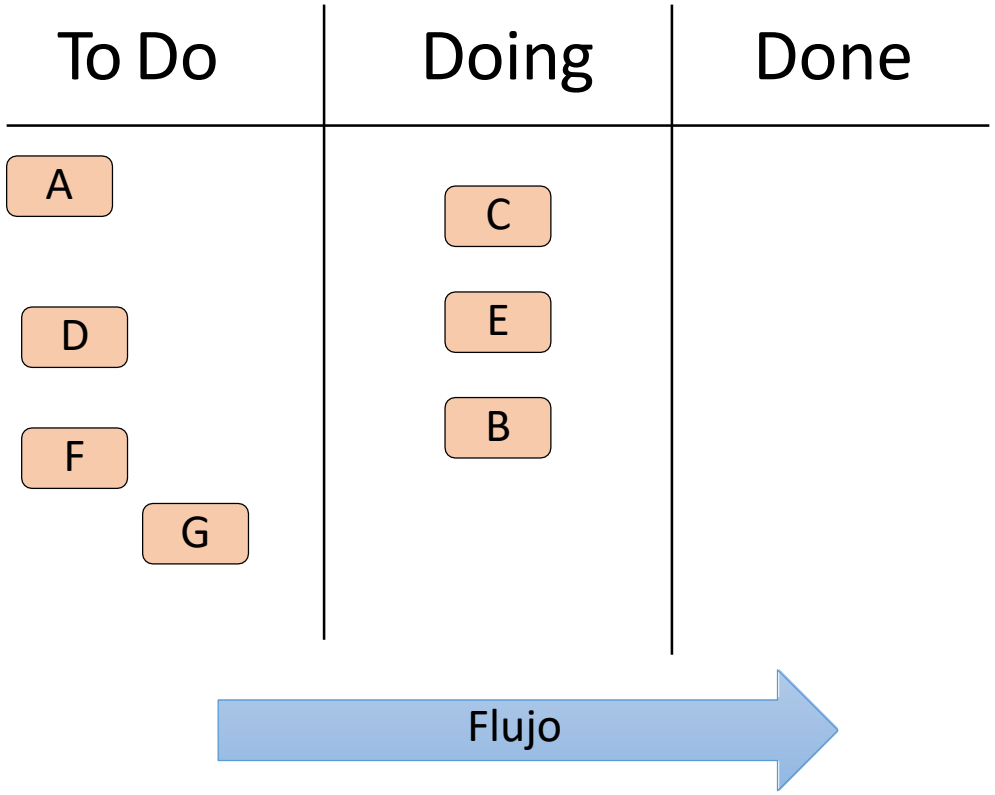
DuocUC®

- **KANBAN**

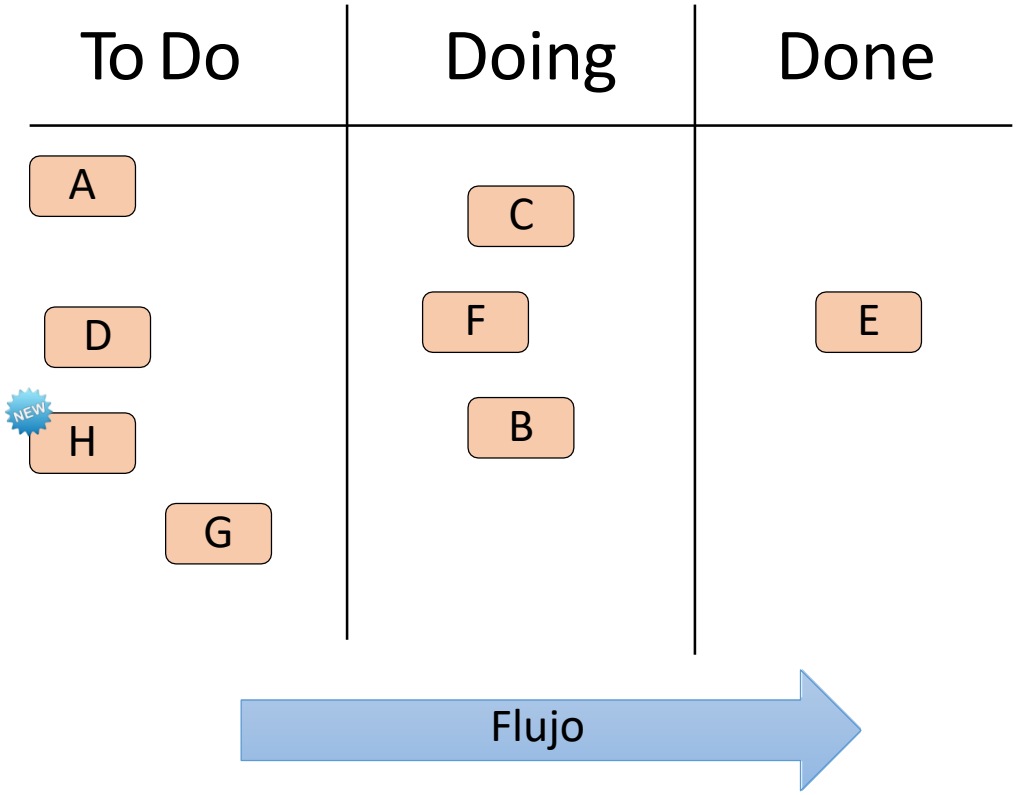
Kanban y kanban

- Kanban es el nombre de un método
- kanban es el nombre del tablero

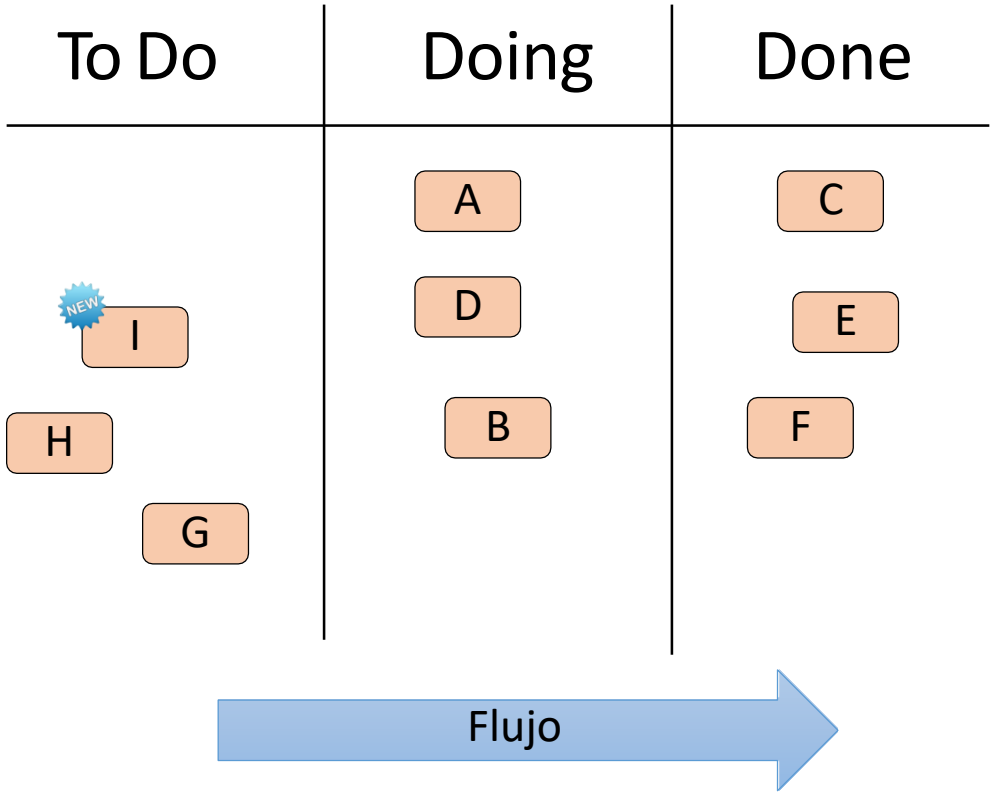
kanban elemental



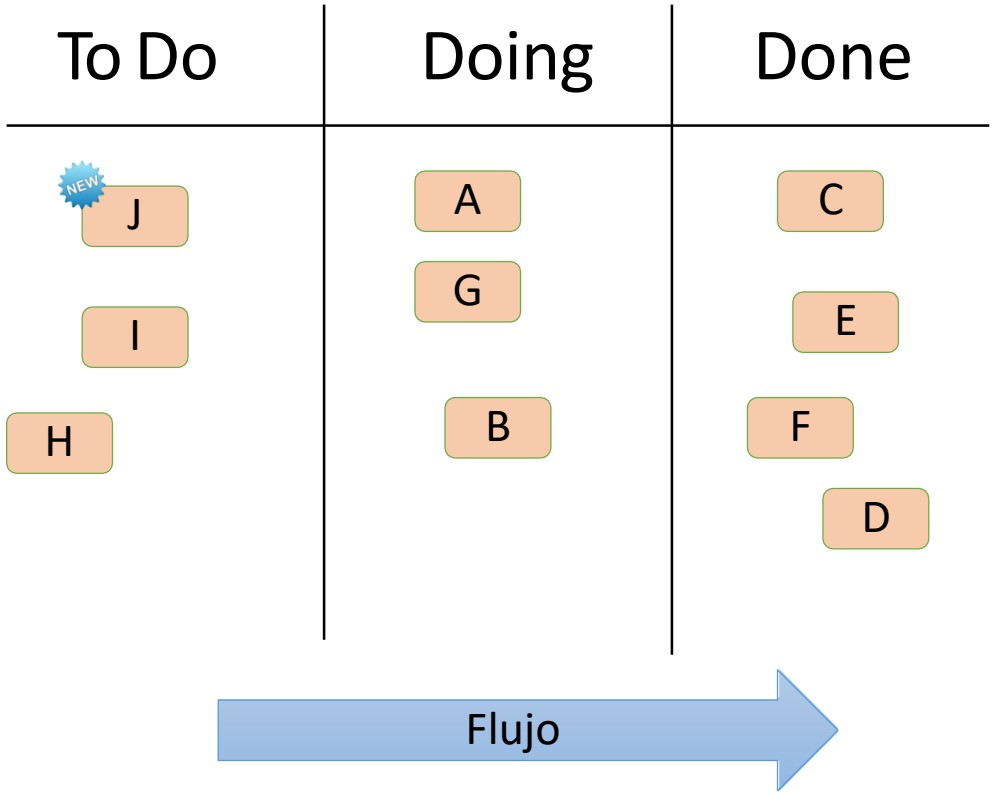
kanban elemental



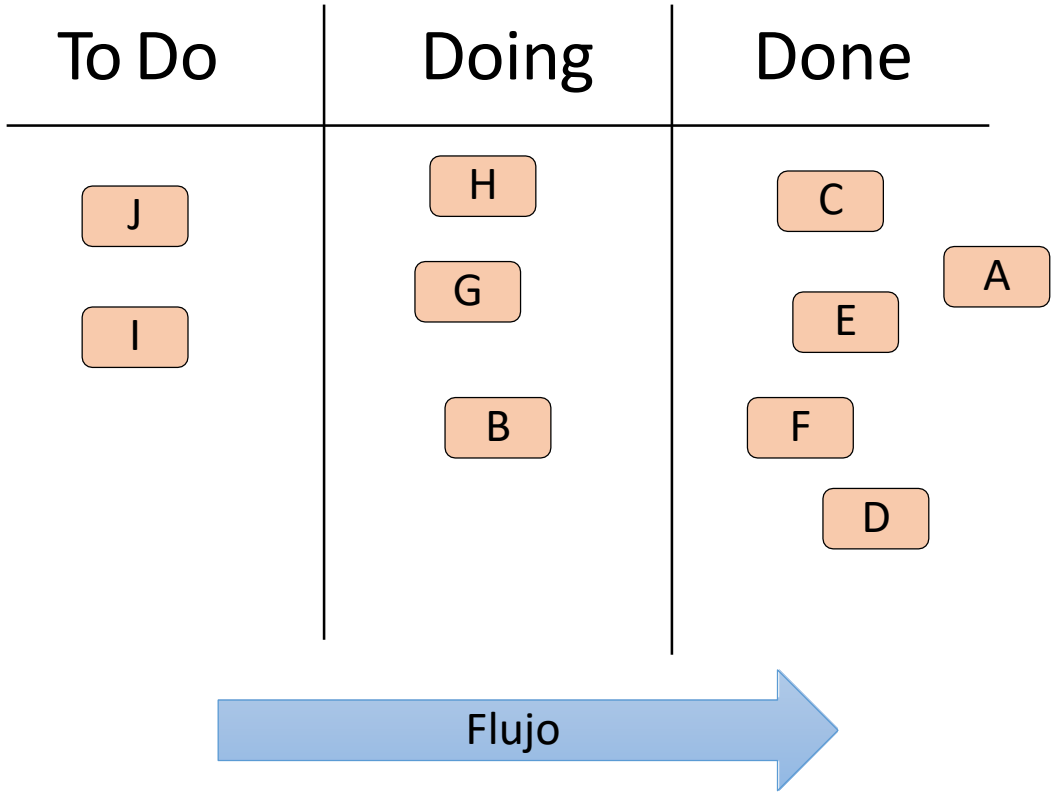
kanban elemental



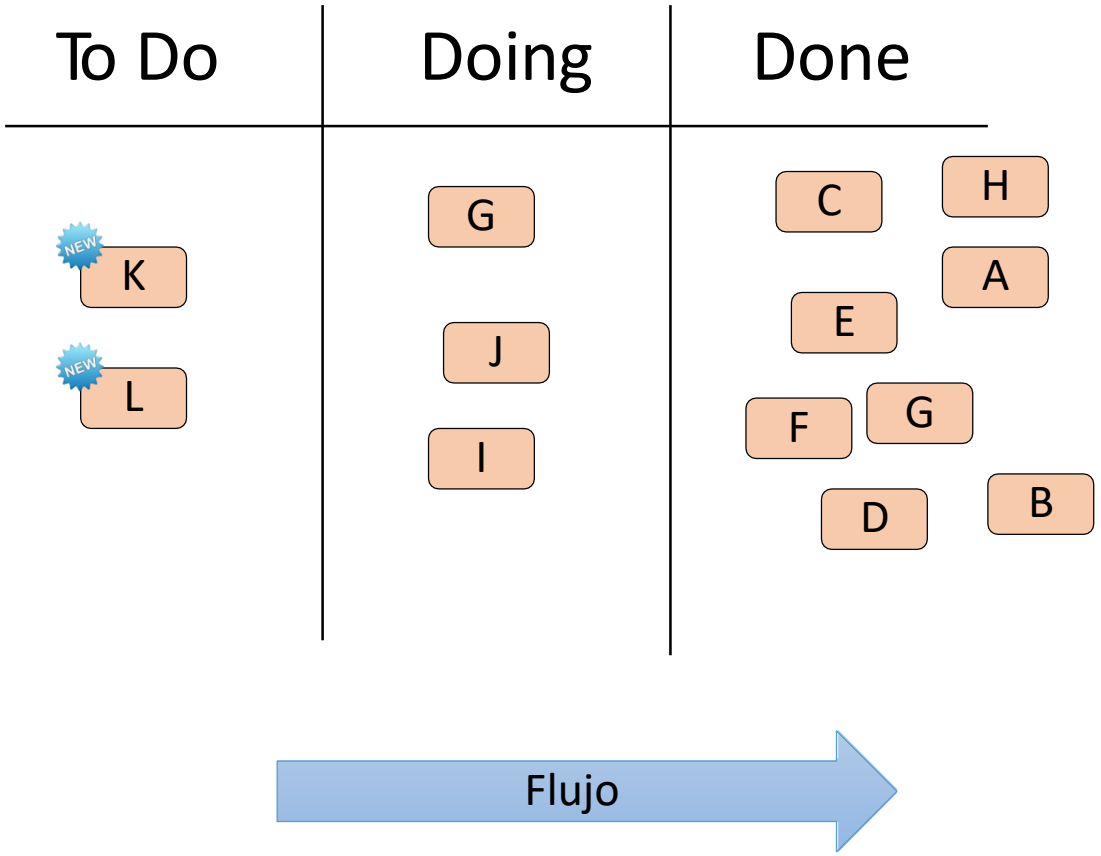
kanban elemental



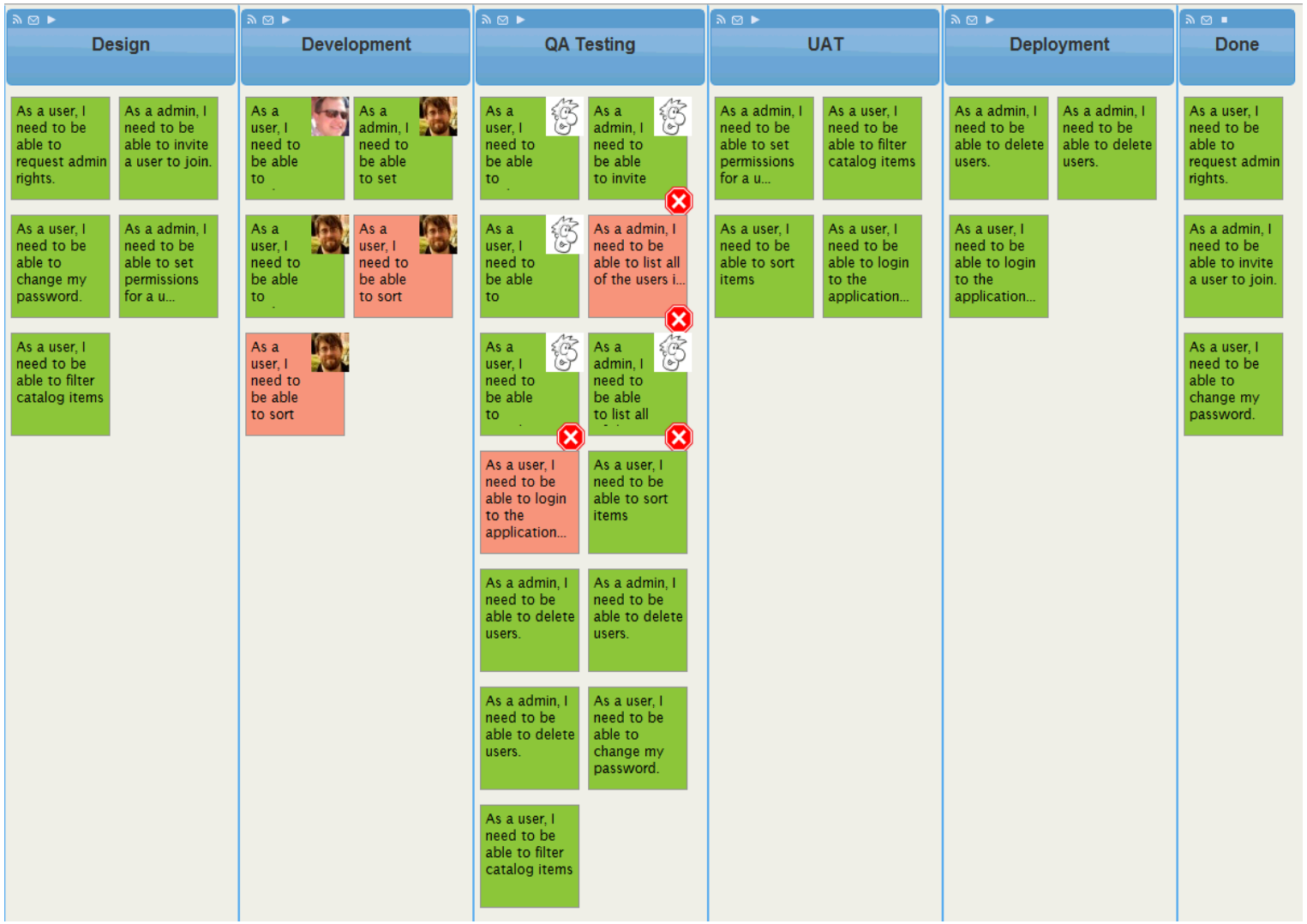
kanban elemental



kanban elemental

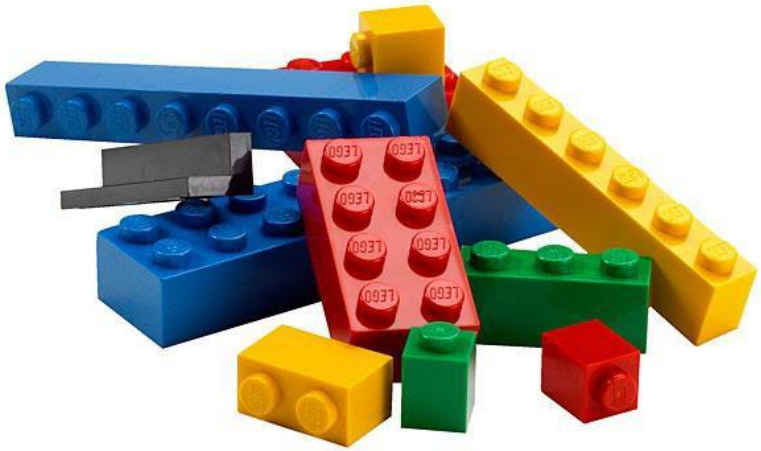


Ejemplo de kanban



Ítems de trabajo

Ítems, Work Items,
Unidades de Trabajo (UTs), Work Units,
Historias de usuario, etc.



¿Qué es un ítem kanban?

Podría ser:

- ❌ Un producto, servicio o proyecto como un todo
- ❌ Un subsistema o módulo
- ❌ Una actividad, por ejemplo, “Programar”
- ✅ Un elemento específico que cambia el producto/servicio, por ejemplo un nuevo requisito, una mejora, una corrección.
- ✅ Una tarea que no cambia el producto/servicio pero que está en su contexto de trabajo, por ejemplo: montar un servidor, estudiar una tecnología, etc.

GRANULARIDAD: Lo importante es intentar que sea un trabajo que el equipo pueda terminar (estando dedicado) en no más de unos días o una semana.

¿Qué hacer con los ítems “grandes” (Épicas)?

1 solo ítem y sin colaboración

1 solo ítem y con colaboración

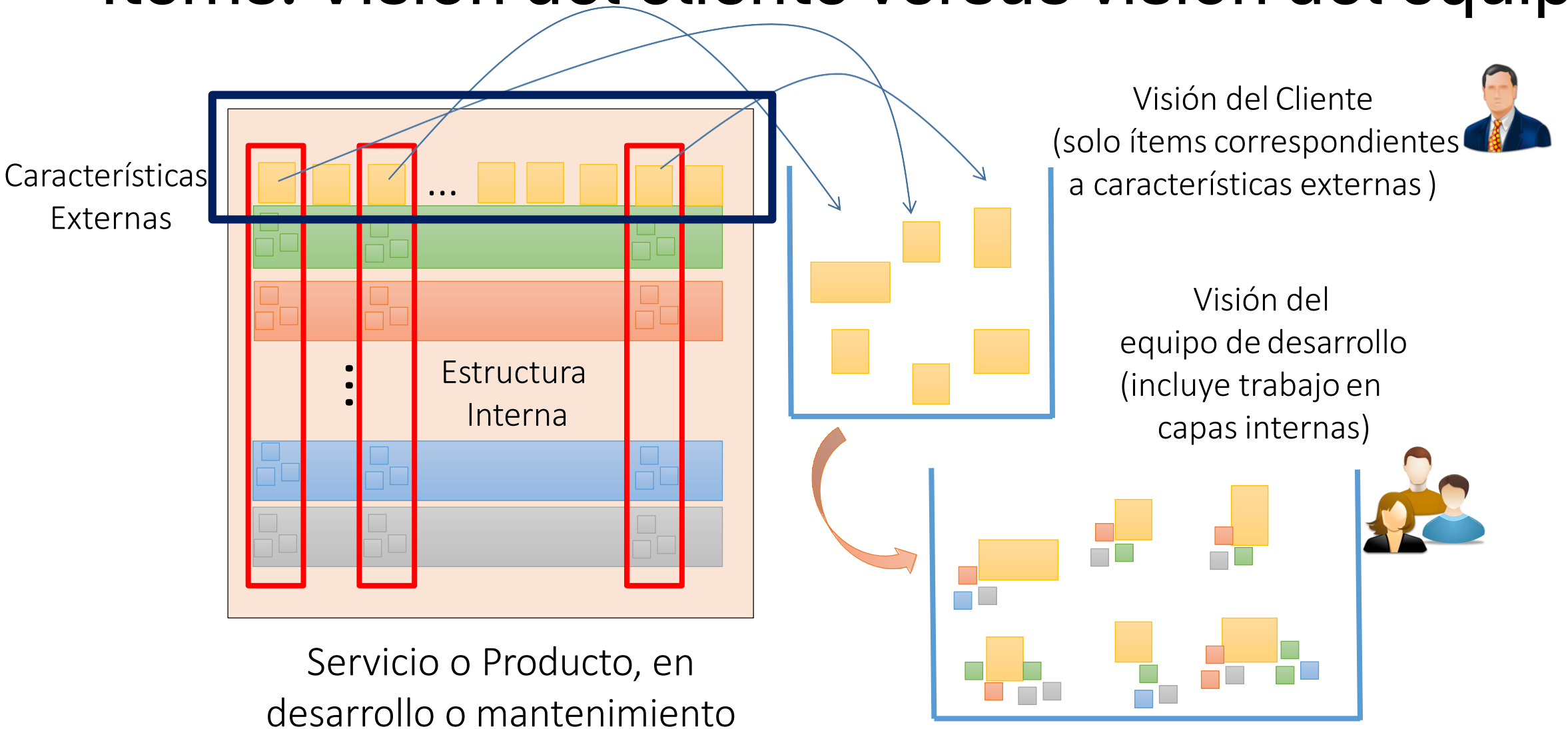
pero …

Dividirlo, varios ítems

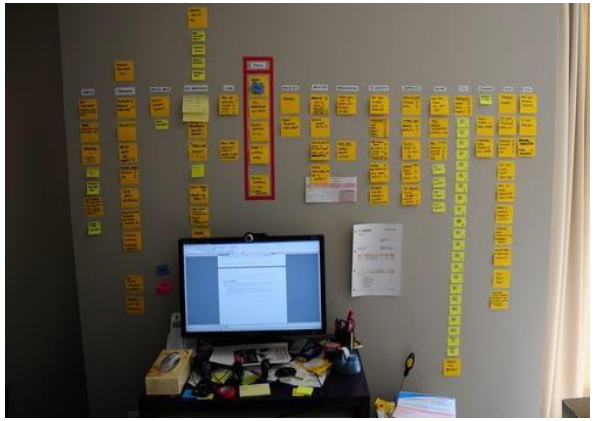
pero …

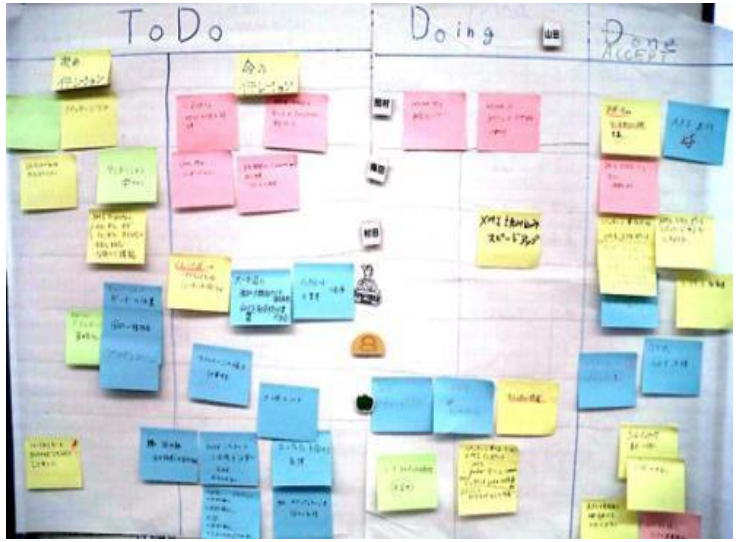
Establecerlo incrementalmente, varios ítems

Ítems: Visión del cliente versus visión del equipo



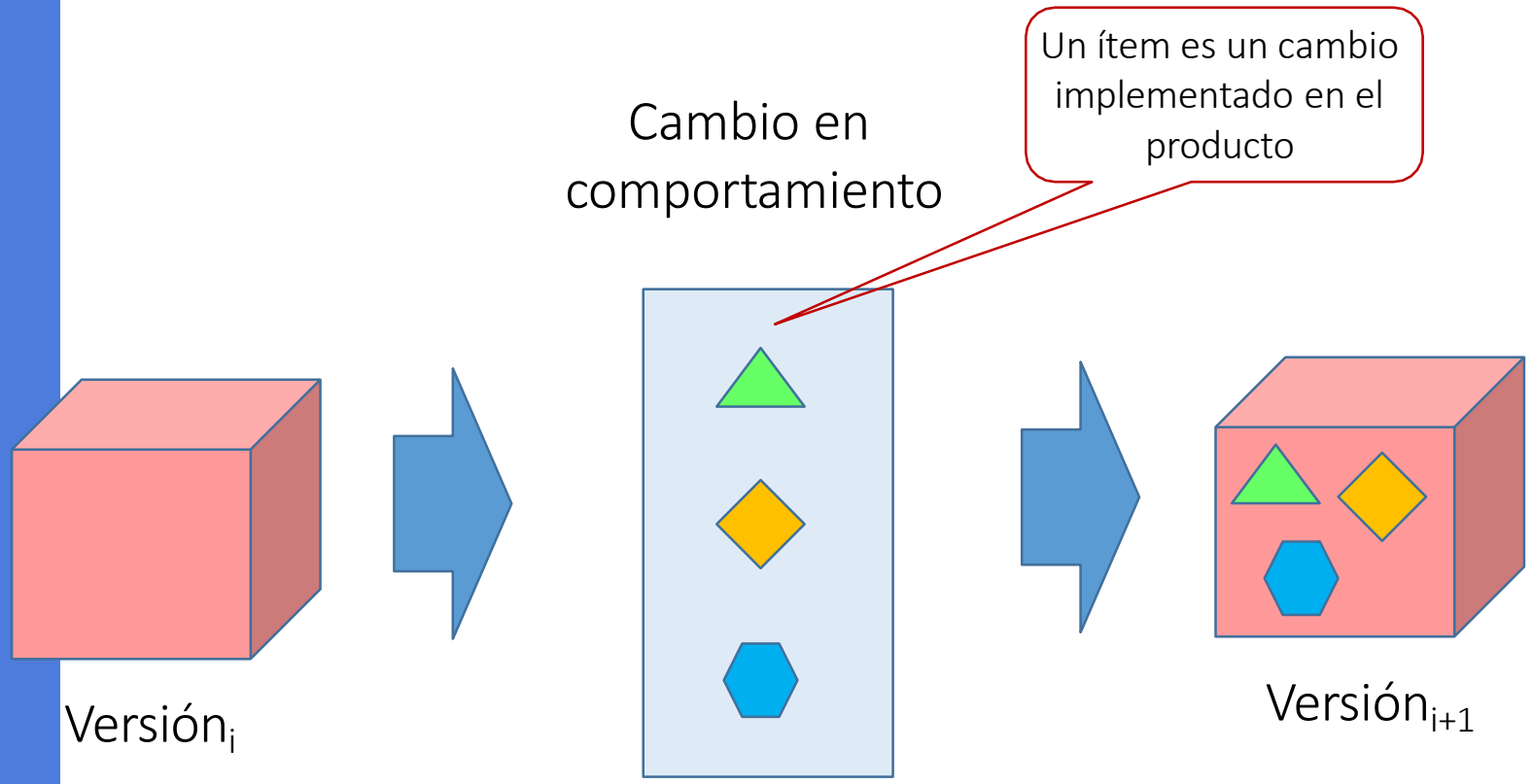
... más ejemplos de kanban





ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

Diseño de tablero kanban



Tipos de cambios en el producto

- Nuevo requisito
- Mejora
- Corrección de defecto

DuocUC®

Actividades involucradas en un cambio

Tipos de ítems de cambio de comportamiento en el producto (software):

- Nuevo Requisito
- Mejora en un requisito existente
- Corrección de un Fallo.

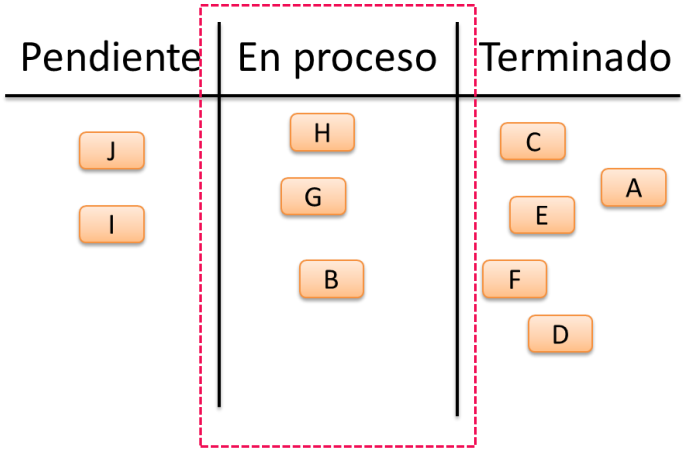
Las actividades esenciales que debe realizar el equipo con estos ítems son: Especificar Requisitos (R), Programar (P) y Aplicar Pruebas de Aceptación (T).

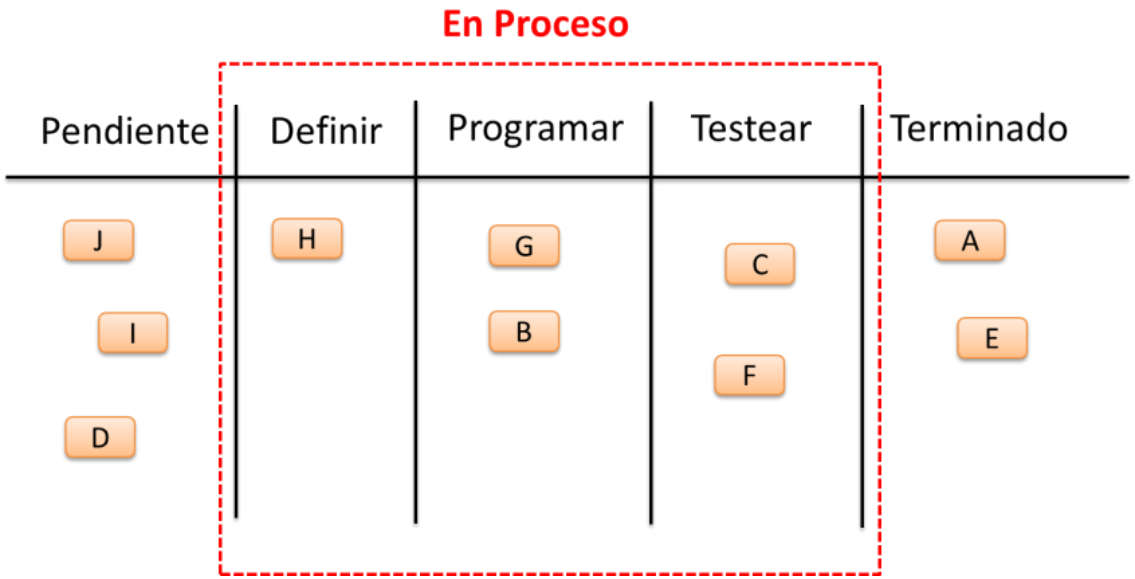
Especificar Requisitos Especificación del cambio de comporta-miento en el sistema (desde la perspectiva del usuario)

Programar Implementación del cambio de comportamiento en el producto

Aplicar Pruebas de Aceptación. Aplicación de Pruebas de Aceptación para verificar la correcta implementación del cambio

Más visibilidad al proceso

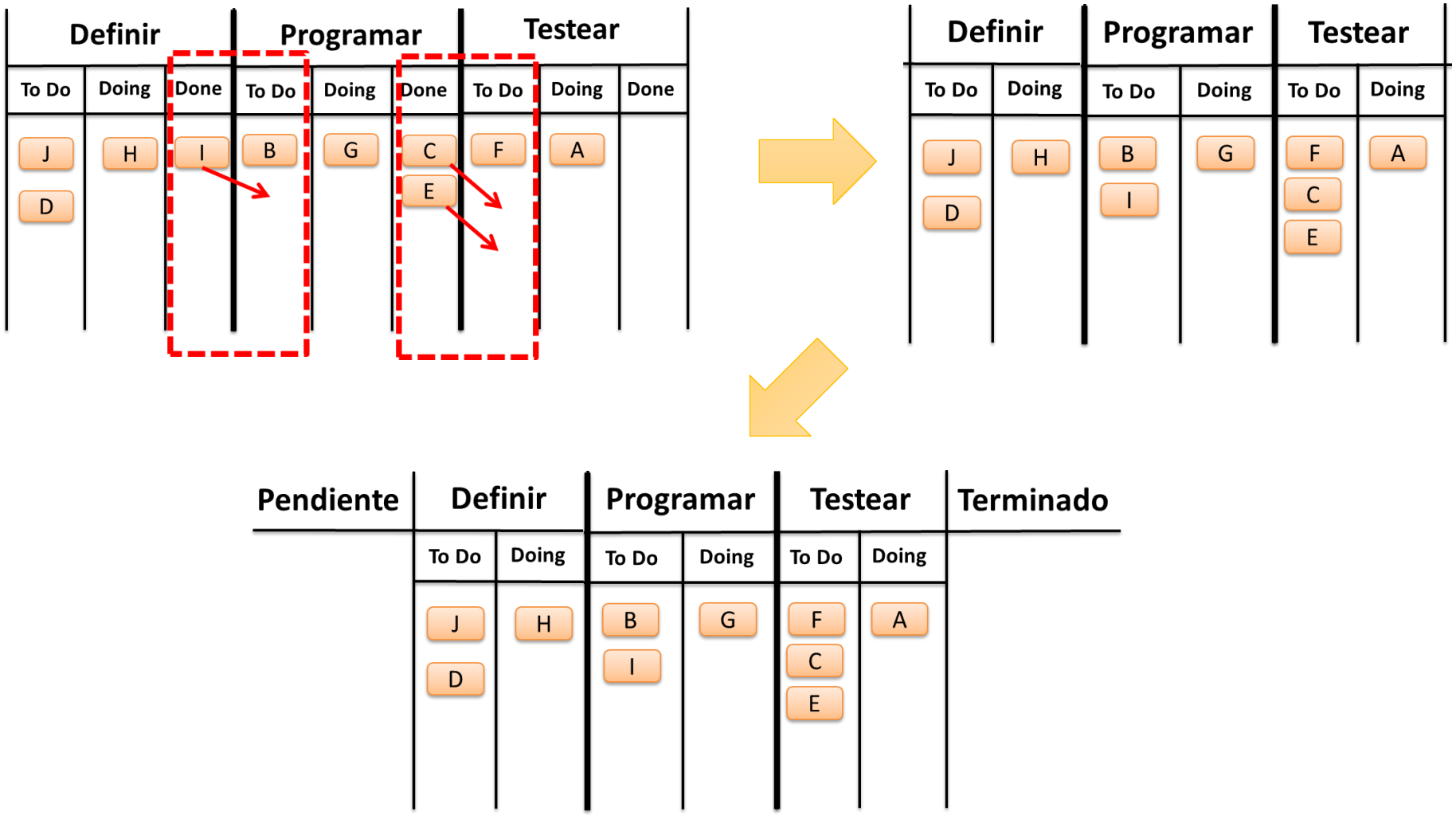




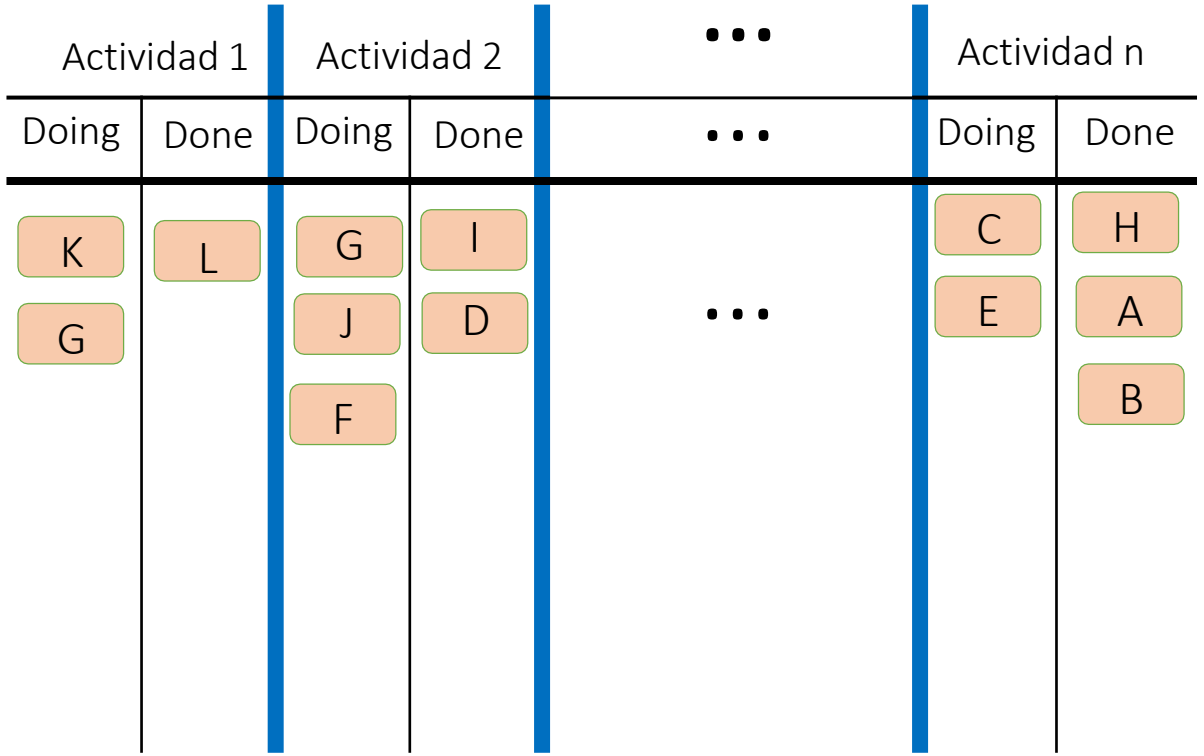
Sub-estados para cada activiad

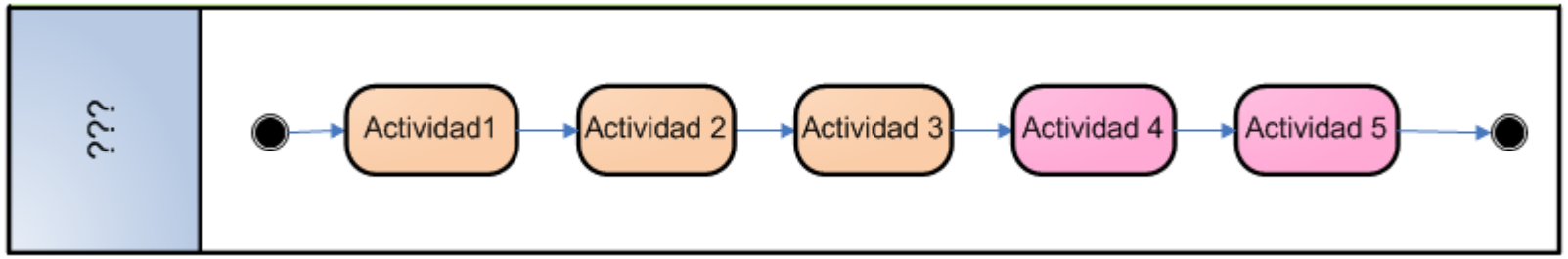
Actividad$_x$		
To Do	Doing	Done
K G	G J F	L

kanban fusionando sub-estados Done y To Do



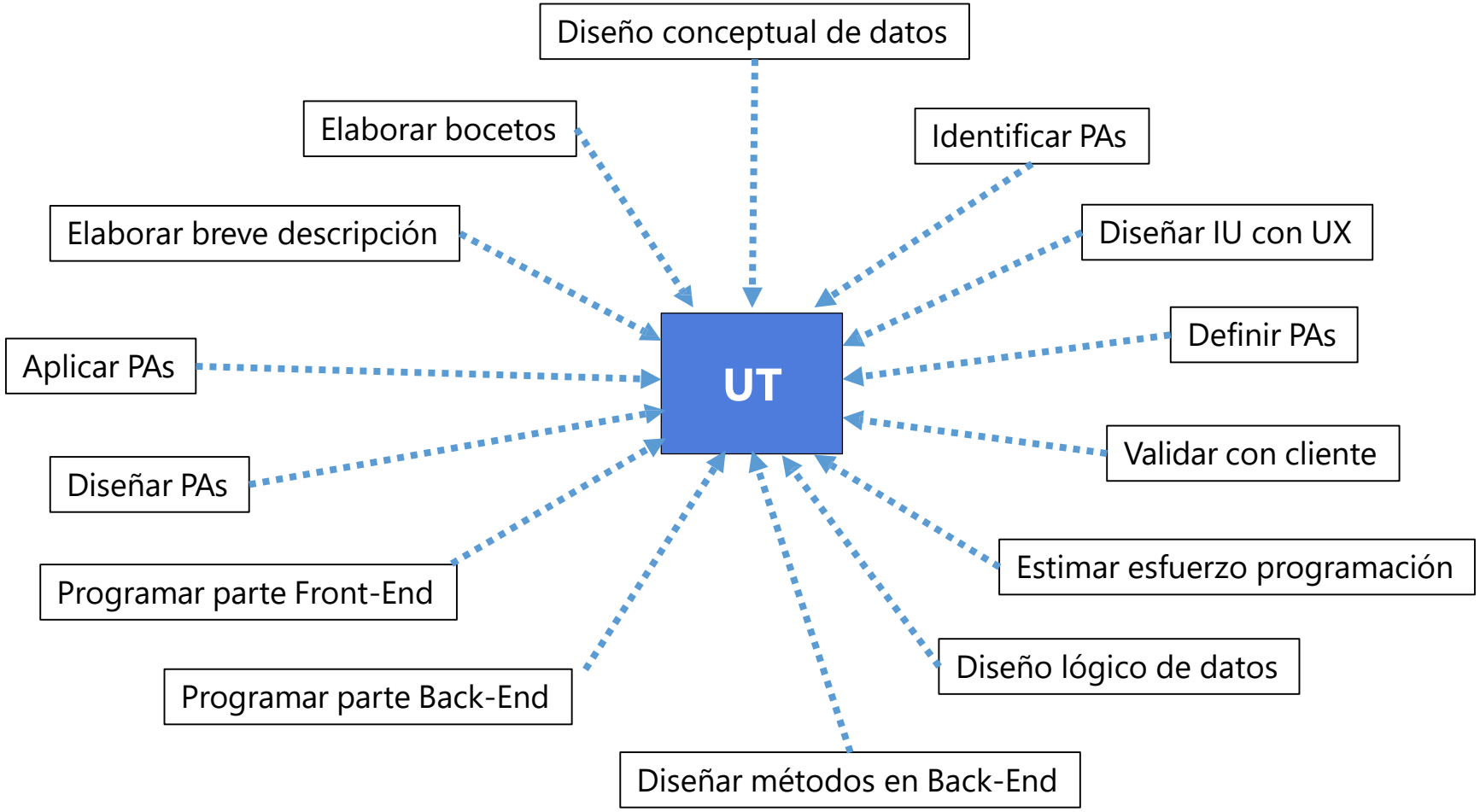
kanban con flujo de trabajo





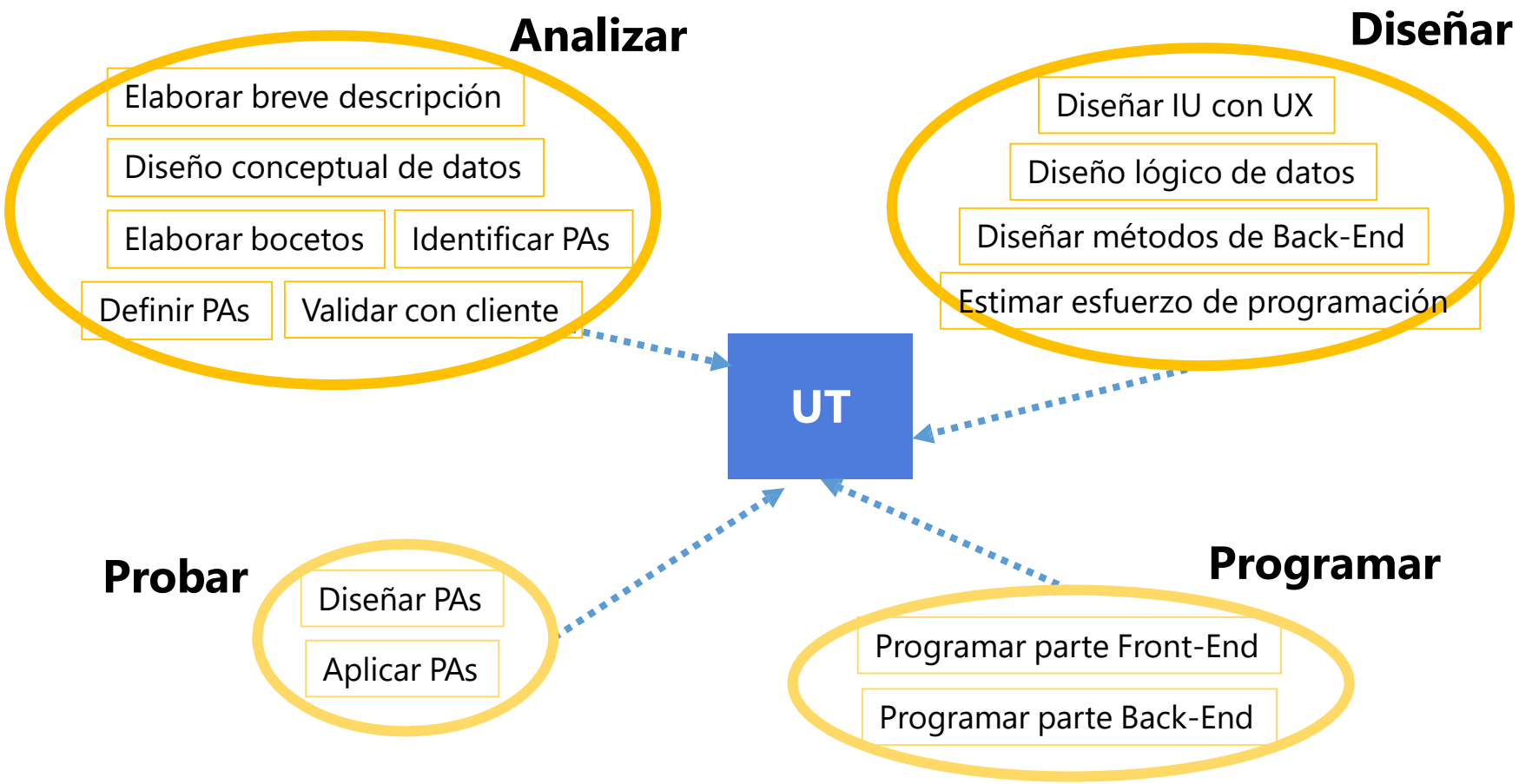
Actividades versus Tareas en un ítem

Ejemplo: posibles tareas en el contexto de un ítem de desarrollo de software



Actividades versus Tareas en un ítem

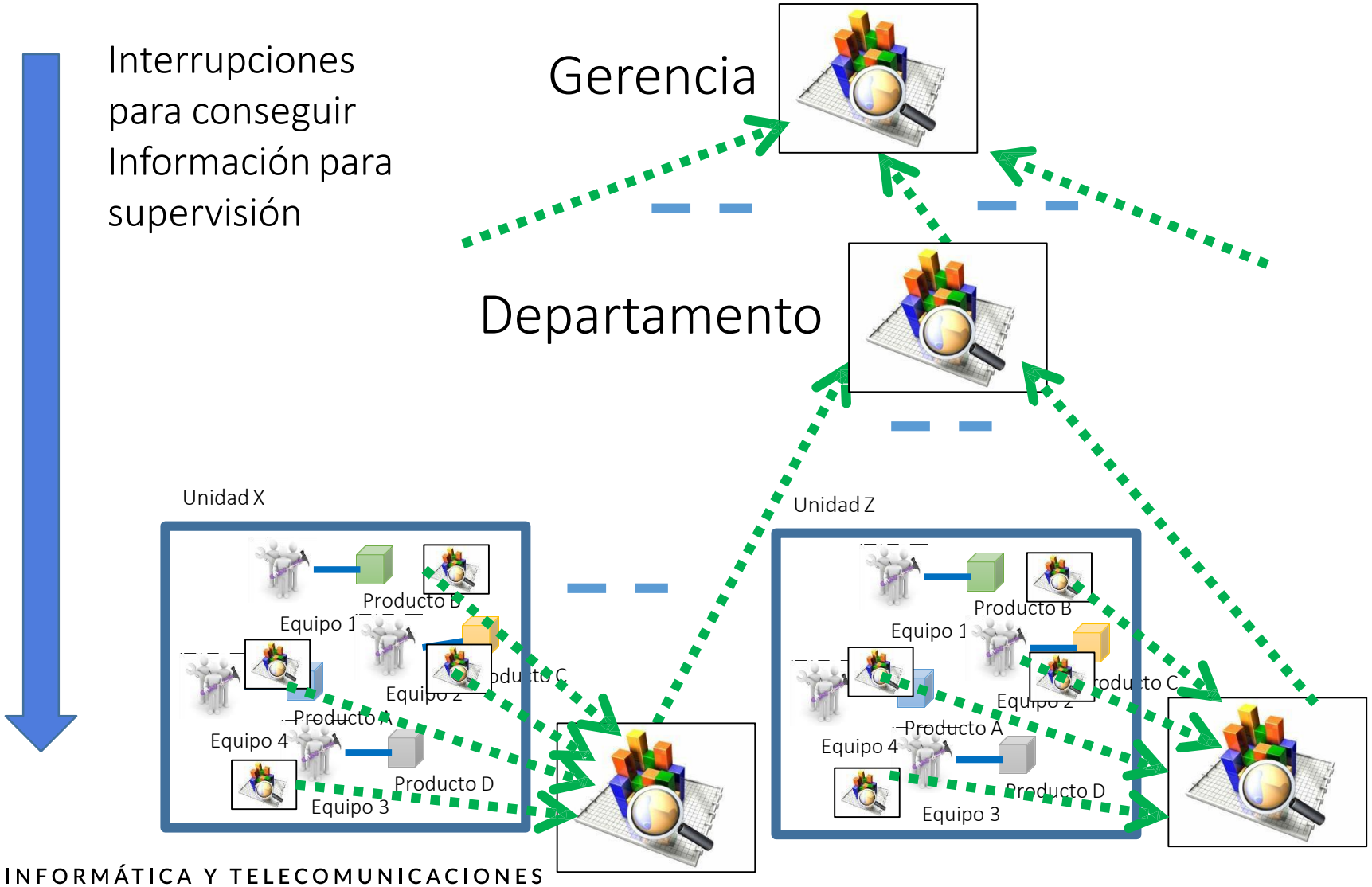
Una posible agrupación y nombres de actividades para el tablero kanban



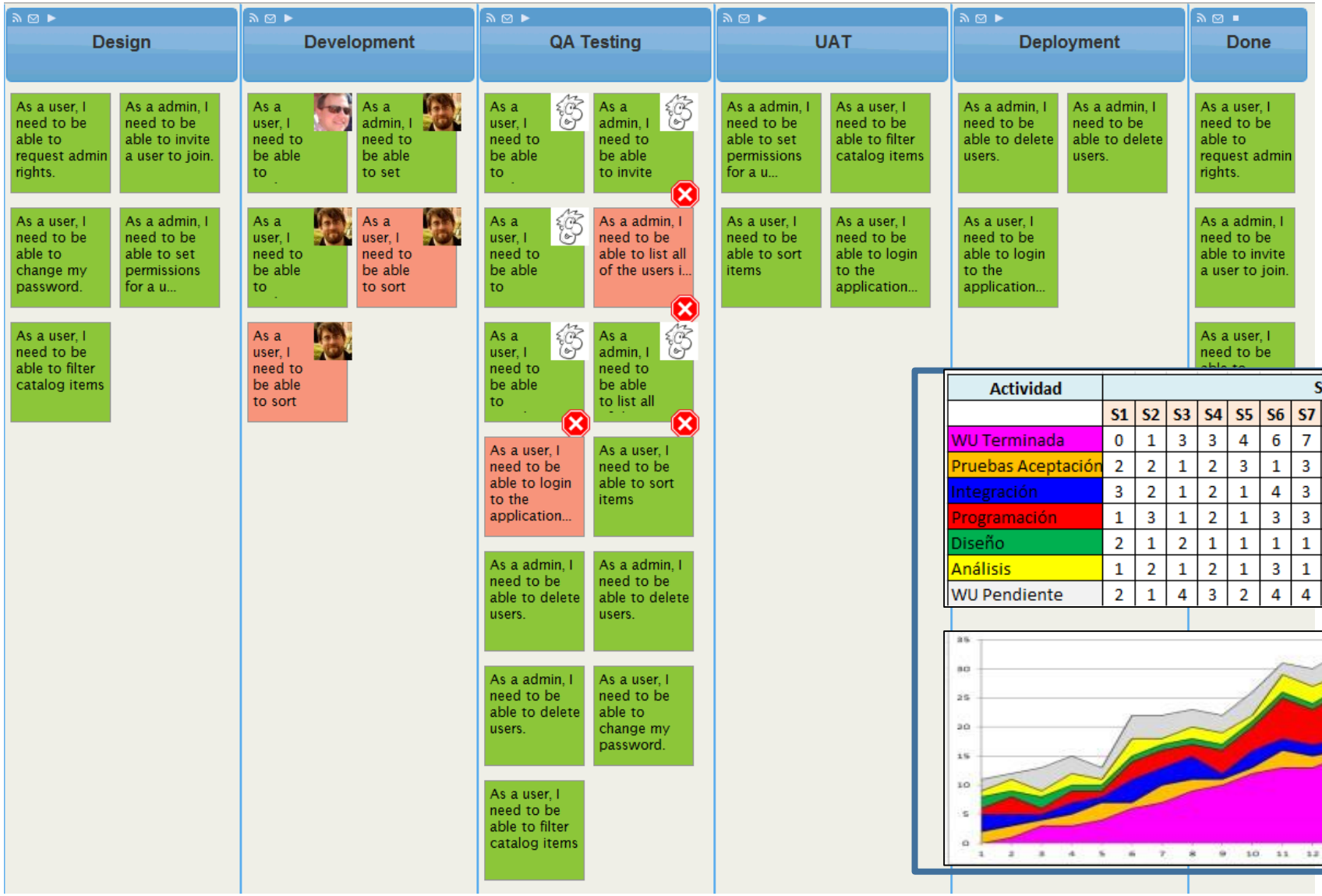
04

Seguimiento Ágil

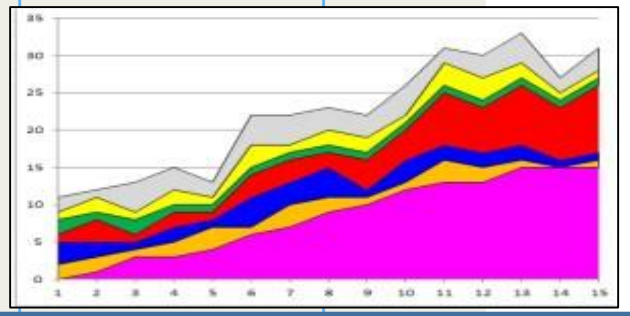
Seguimiento tradicional



Seguimiento básico: Tablero kanban

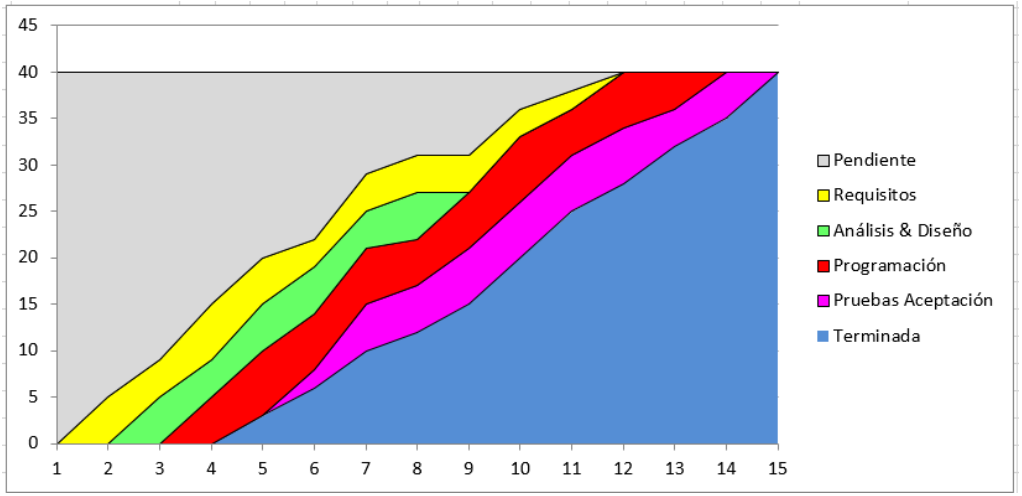


Actividad	Snapshot									
	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10
WU Terminada	0	1	3	3	4	6	7	9	10	12
Pruebas Aceptación	2	2	1	2	3	1	3	2	1	1
Integración	3	2	1	2	1	4	3	4	1	3
Programación	1	3	1	2	1	3	3	2	4	4
Diseño	2	1	2	1	1	1	1	1	1	1
Análisis	1	2	1	2	1	3	1	2	2	1
WU Pendiente	2	1	4	3	2	4	4	3	3	4



Actividad: kanban y Diagrama de Flujo Acumulado

	Snapshot														
Actividad	**S1**	**S2**	**S3**	**S4**	**S5**	**S6**	**S7**	**S8**	**S9**	**S10**	**S11**	**S12**	**S13**	**S14**	**S15**
Terminada	0	0	0	0	3	6	10	12	15	20	25	28	32	35	40
Pruebas Aceptación	0	0	0	0	0	2	5	5	6	6	6	6	4	5	0
Programación	0	0	0	5	7	6	6	5	6	7	5	6	4	0	0
Análisis & Diseño	0	0	5	4	5	5	4	5	0	0	0	0	0	0	0
Requisitos	0	5	4	6	5	3	4	4	4	3	2	0	0	0	0
Pendiente	40	35	31	25	20	18	11	9	9	4	2	0	0	0	0



Gráficas para seguimiento ágil.xlsx. (s/f). Google Docs. Recuperado el 1 de julio de 2024, de https://drive.google.com/file/d/148up2Q1L7n4voAhfOAdmaD4RTvmwCVr3/view?usp=sharing&usp=embed_facebook

Desafío

• Aplicación de métricas de flujo

Suponga que tenemos dos equipos de trabajo que atienden incidencias de infraestructura (fallos de conexión, caídas de servidores, permisos de acceso, ataques de seguridad, etc.) y que nos presentan las siguientes métricas:

	Production Rate (*)	Cycle Time	Lead Time
Equipo 1	3.5	3 días	5 días
Equipo 2	2.8	4 días	6 días

(*) en incidencias/semana

¿Qué equipo muestra mejor desempeño? ¿Bajo qué supuesto es válida la comparación?

Conseguiremos cumplir lo acordado?

¿Qué indicadores utilizar para realizar el seguimiento?

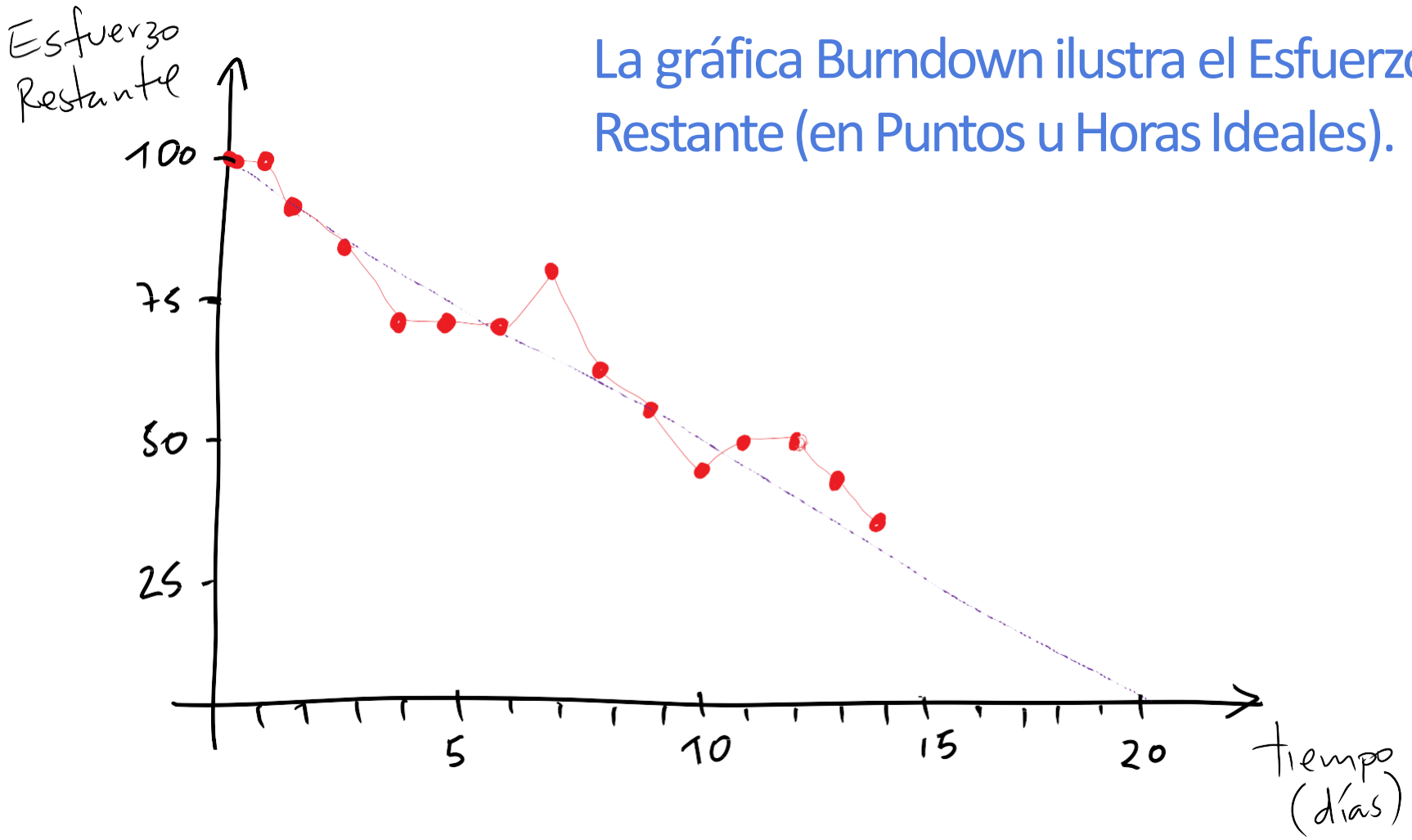
Esfuerzo restante

¿Cuántas horas ideales (o Puntos) suma el trabajo no terminado?

Capacidad disponible

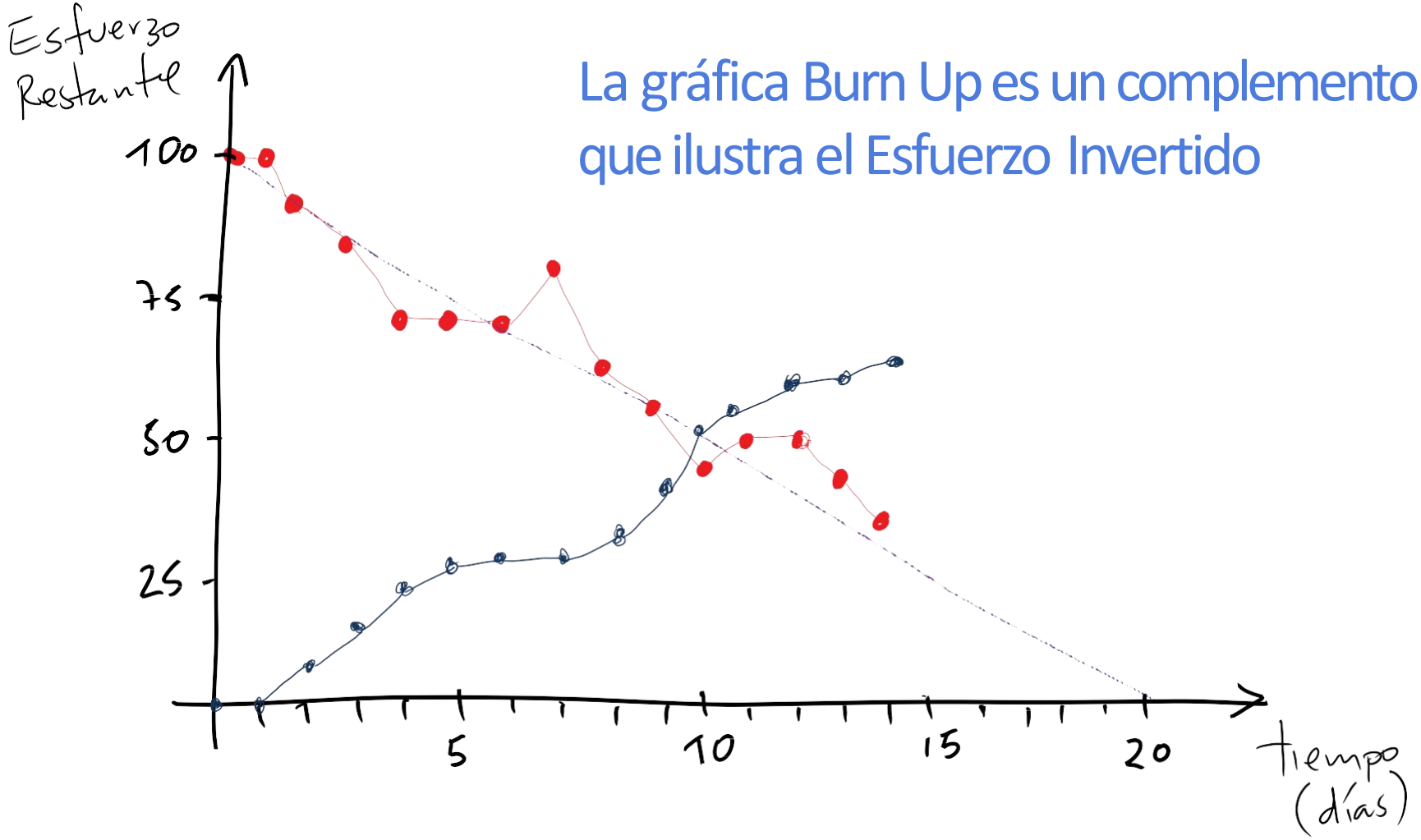
¿Cuántas horas ideales (o Puntos) puede abordar el equipo en el tiempo disponible?

Gráficas Burndown



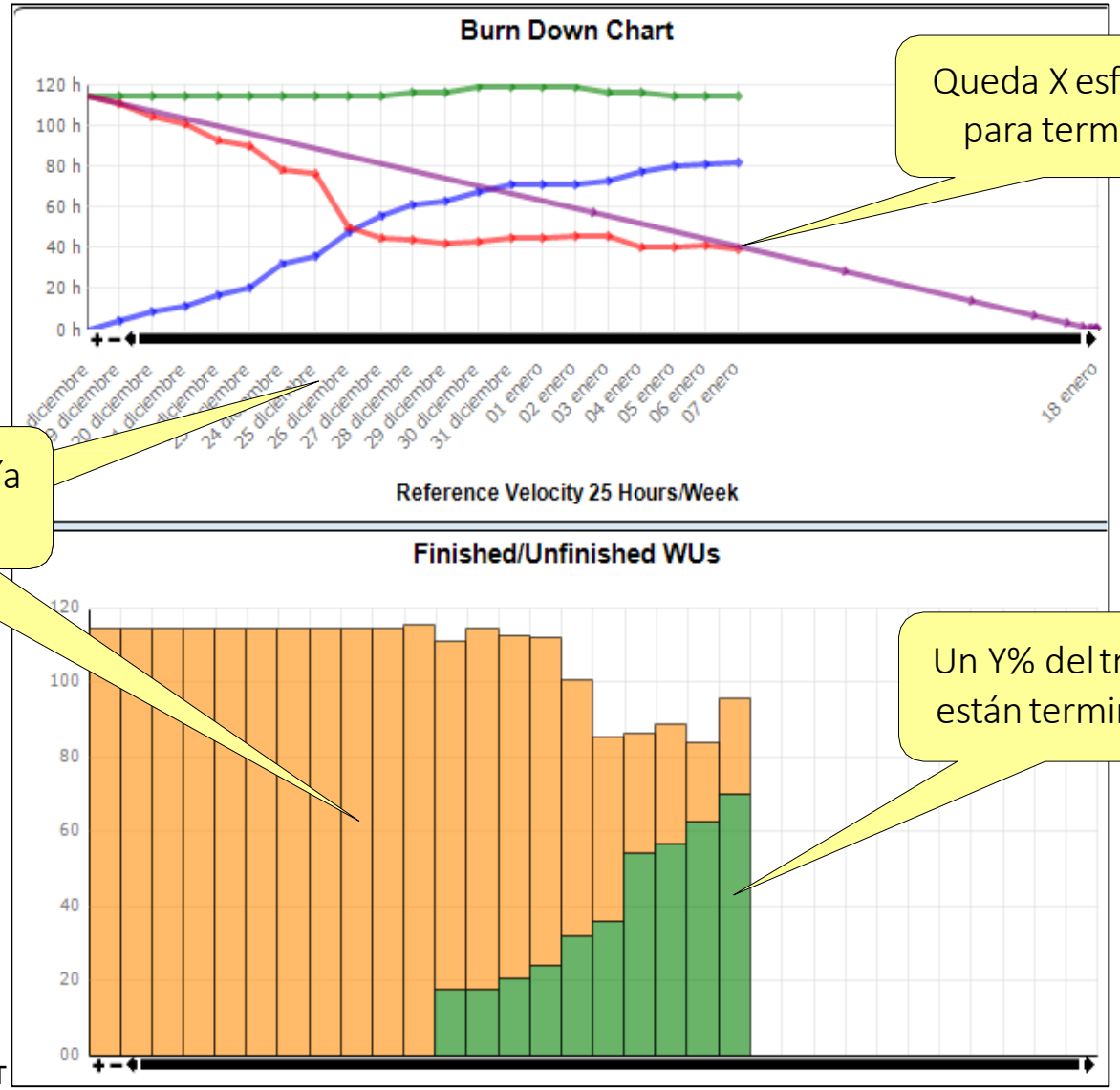
La gráfica Burndown ilustra el Esfuerzo Restante (en Puntos u Horas Ideales).

Gráficas Burndown



La gráfica Burn Up es un complemento que ilustra el Esfuerzo Invertido

Seguimiento de Sprints



Queda X esfuerzo para terminar

¡Seguimiento día a día!

Un Y% del trabajo están terminadas

Tablero kanban + sprint

Letelier, P. (s/f). *Agility at work*. Blogspot.com. Recuperado el 1 de julio de 2024, de http://agilismoatwork.blogspot.com/2013/05/kanban-o-scrum-esta-no-es-la-cuestion.html

Tablero kanban + sprint

Es conveniente usar Sprints en una o más de las siguientes situaciones:

- El trabajo es planificable, es decir, se puede y quiere hacer una previsión o compromiso del trabajo que se terminará en un período (corto).
- Se desea hacer un seguimiento más preciso basado en estimaciones del trabajo y en la capacidad del equipo.
- Se quiere agrupar trabajo para una entrega.

Letelier, P. (s/f). *Agility at work*. Blogspot.com. Recuperado el 1 de julio de 2024, de http://agilismoatwork.blogspot.com/2017/06/sprints-welcome-to-real-world.html

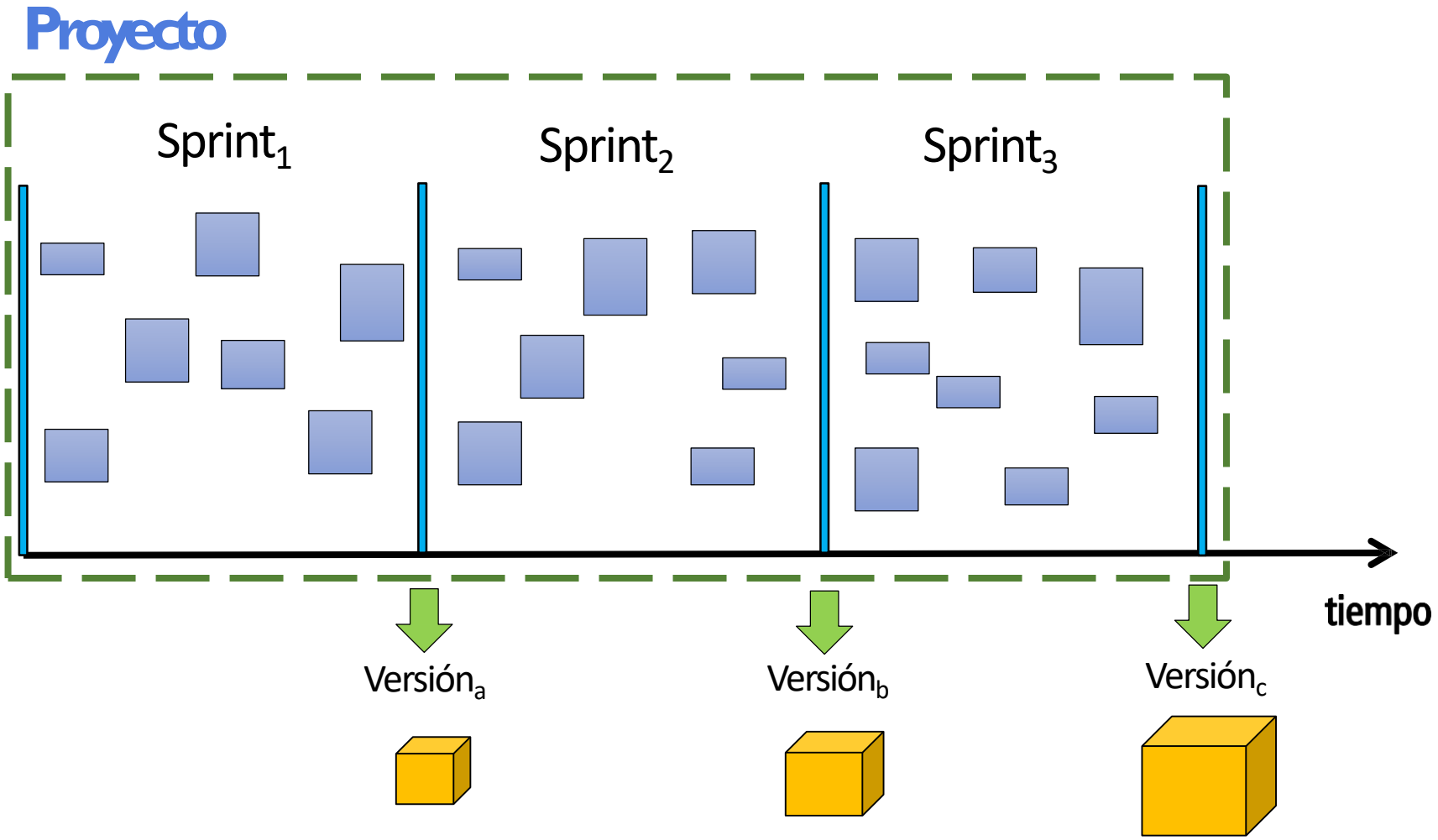
ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

¿Qué sería un "Proyecto" en un enfoque ágil?

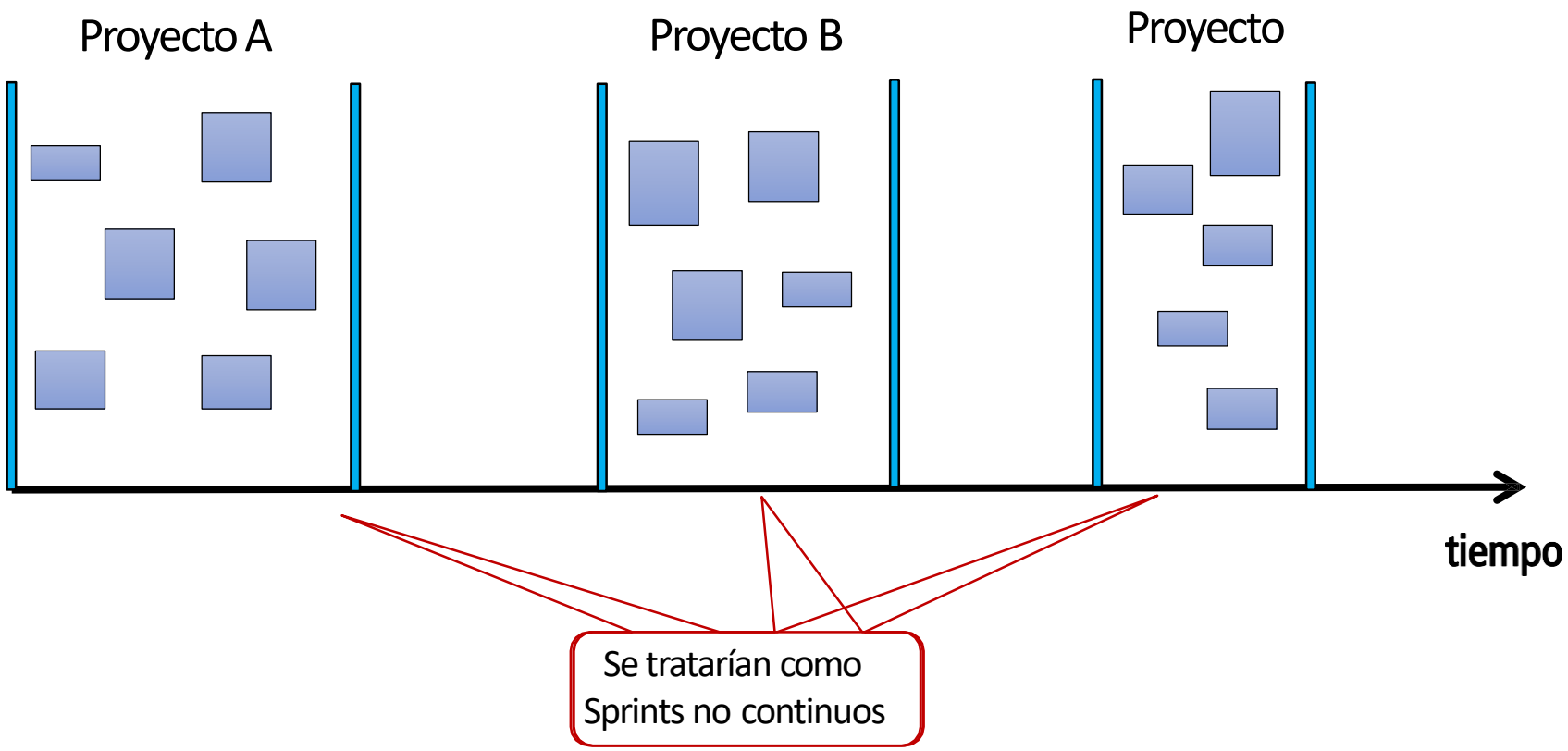
Letelier, P. (s/f). *Agility at work*. Blogspot.com. Recuperado el 1 de julio de 2024, de http://agilismoatwork.blogspot.com/2011/11/entrega-release-iteracion-sprint.html

DuocUC®

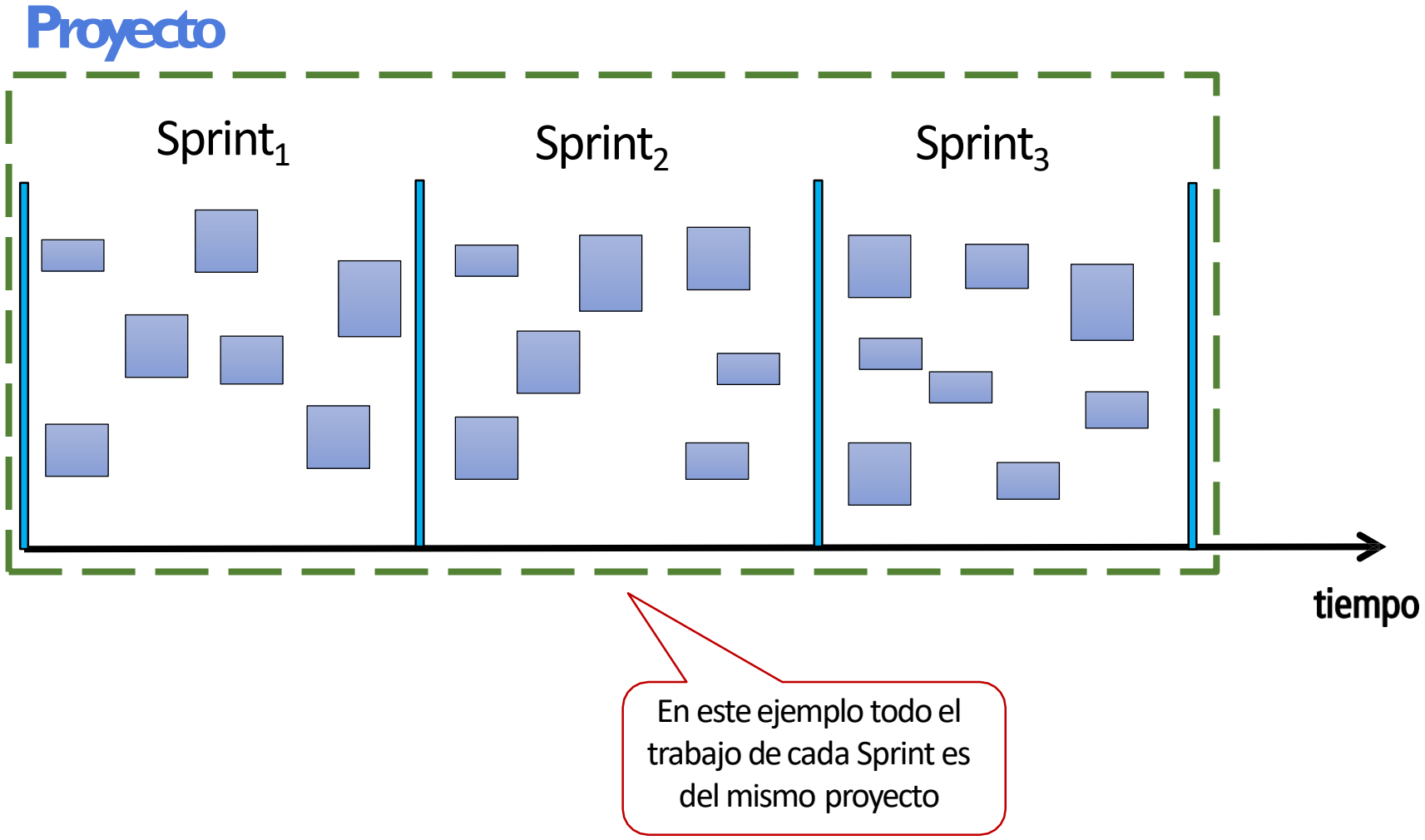
Sprints de un Proyecto



Proyectos de corta duración en un mismo producto



Sprints de un Proyecto en un Producto



01

Gestión de Calidad de Software

• Gestión de Calidad de Software

- El objetivo final de la Ingeniería de Software es producir software de **alta calidad**

- La **calidad** abarca todo el proceso y está determinada por factores directos e indirectos.

- La **calidad** es un concepto complejo y multifacético, que se puede describir desde muchos puntos de vista.

Nunca comprometer 0 defectos

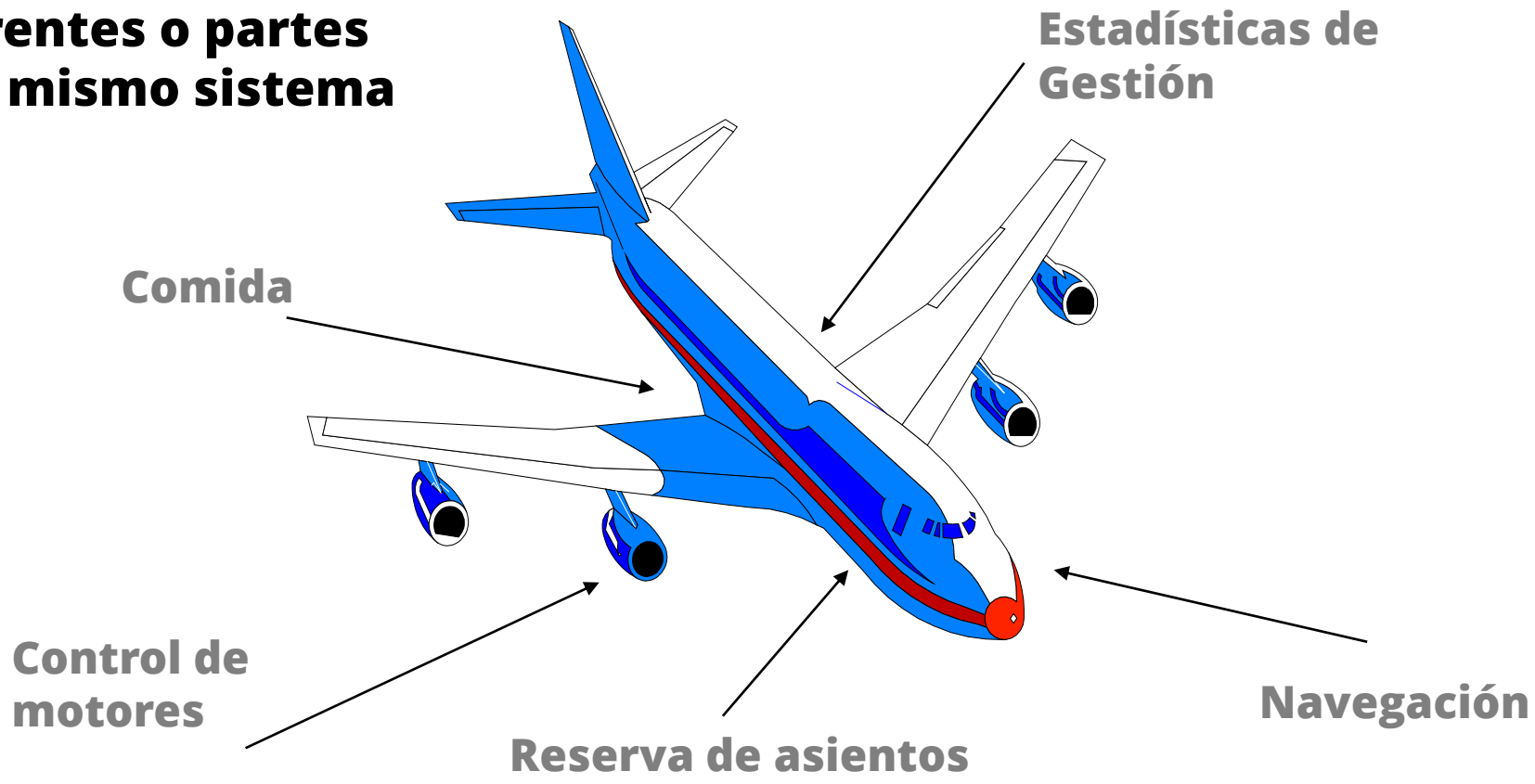
• Diversos puntos de vista de la calidad

Programador

Cliente

• Diversas necesidades de la calidad

Sistemas diferentes o partes diferentes del mismo sistema



No aplicar cualidades críticas a sistemas totales

• Vistas de la Calidad de Software

Vista trascendental

- Se reconoce la calidad pero no se define

Vista de usuario

- Se ajusta al propósito

Vista de fabricación

- Conformidad con la especificación

Vista de producto

- Características inherentes del producto

Vista basada en valor

- Valor monetario

• Vista Trascendental

- Concibe la calidad como algo que puede reconocerse pero es **difícil de definir**. Aquí la calidad es algo que se puede reconocer a través de la experiencia, pero que no se define de alguna forma manejable.

- La calidad se considera **algo ideal**, demasiado complejo para prestarse a una definición precisa.

- El grado de calidad o estándar de excelencia es subjetivo, no se puede analizar y no se puede medir.

Esta vista no es adecuada para el Software

• Vista de Usuario

- Percibe la calidad como adecuación a un **propósito**. Según este punto de vista, al evaluar la calidad del producto, uno debe hacer la pregunta clave "¿**El producto satisface las necesidades y expectativas del usuario**?"

- Desde este punto de vista, al usuario le preocupa si un producto **es apto para su uso o no**. La calidad no solo se ve en términos de lo que un producto puede ofrecer, sino que también se ve influenciada por las disposiciones del servicio en el contrato de venta.

- Se apoya en modelos de confiabilidad, rendimiento y laboratorios de usabilidad

• Vista de Fabricación

- Aquí la calidad se entiende como **conformidad con las especificaciones**. El nivel de calidad de un producto está determinado por el grado en que el producto cumple con sus especificaciones.

- Cualquier desviación de los requisitos establecidos se considera una **reducción de la calidad del producto**. La visión de fabricación tiene su génesis en los sectores productivos, como los sectores del automóvil y la electrónica.

- Se focaliza en el **proceso de software**, dando énfasis a hacerlo "bien a la primera" , minimizar el retrabajo antes y después de la entrega.

- La evaluación de la calidad es prácticamente independiente del producto, por ejemplo, con la aplicación de CMMI o ISO 9001.

• Vista de Producto

- En este caso, la calidad se considera vinculada a las **características inherentes del producto**. Las cualidades internas de un producto determinan sus cualidades externas.

- La vista del producto es atractiva porque brinda la oportunidad de explorar las relaciones casuales entre las propiedades internas y las cualidades externas de un producto.

- Un ejemplo de la visión del producto de la calidad del software es que el alto grado de modularidad, que es una propiedad interna, hace que un software sea comprobable y mantenible.

Poca evidencia que demuestre que las medidas internas afectan las características externas.

• Vista basada en Valor

Visión de jefes de proyecto

- Balancear requerimientos con presupuestos y plazos
- Manejar conflictos originados en cambios en los requerimientos
- Apoyar ciertas estrategias de desarrollo como diseñar al costo y *adecuación* de los requerimientos

Visión del comprador

- Valor por el dinero para una organización particular
- Se basa en evaluación de beneficios del producto
- Independiente de características internas del producto

ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

¡1 Cosa!
¿Qué recuerdas?

¿Qué rescato o creo que es importante sobre las vistas de la Calidad de Software?

DuocUC®

• Aseguramiento de Calidad de Software (SQA)

***Software Quality Assurance (SQA)* :** Acciones sistemáticas y planificadas requeridas para asegurar la calidad de software.

- **Objetivos**: Planificar, desarrollar y controlar el proceso de **verificación** y **validación**.

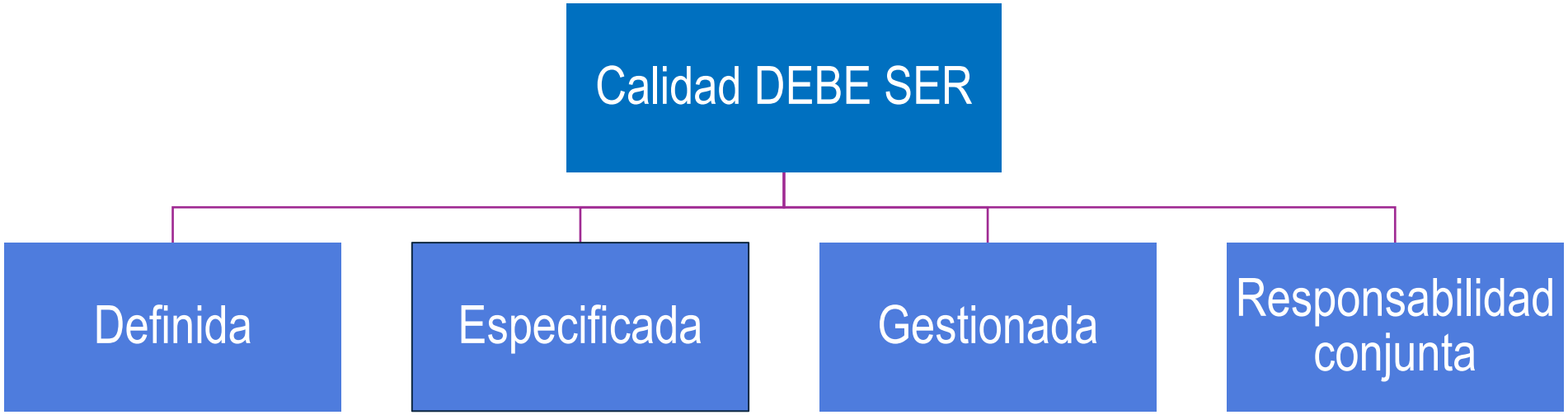
- **Actividades**: Aplicación de métodos, **revisiones e inspecciones**, pruebas, aplicación de estándares, control de cambios, mediciones, registro.

• Aseguramiento de Calidad de Software (SQA)

- **Verificación** -> ¿estamos construyendo el producto correctamente? (*are we building the product right?*)

- **Validación** -> ¿estamos construyendo el producto correcto? (*are we building the right product?*).

• Principios de SQA



• Medición de la Calidad

- Las visiones de calidad afectan la definición
- Definiciones afectan las mediciones
- Definición ISO

Calidad: la totalidad de características de un producto de software que determinan su ***capacidad*** para satisfacer necesidades establecidas e implícitas

.... necesidades ***implícitas*** son peligrosas

• Medición de la Calidad

- **Software** es un concepto nuevo
  - normalmente, *el* ***negocio*** no sabe de software
  - muchas veces, la gente de software no comprende *el negocio*
  - **entonces, hacer todo explícito!**

- **Ejemplo** : La visión del usuario de todos los aspectos de un producto

  - funciones (¿qué hace?)
  - comportamiento operacional (¿cómo lo hace?)
  - restricciones (¿cuánto cuesta?, ¿qué plataformas? etc.)

• Vista de Usuario - Mediciones

- **Aspectos operacionales**
  - Confiablidad (tasa de falla, o tiempo entre fallas)
  - Desempeño (tiempo para desarrollar tareas)

- Muchas ***preocupaciones*** de los usuarios expresadas en términos de conceptos abstractos
  - *Usabilidad, mantenibilidad*

- **Se sugiere**
  - refinar cada concepto en aspectos medibles diferentes.
  - *Ej: usabilidad*
    - Tiempo de aprendizaje (horas para lograr cierto nivel de competencia para un usuario típico)
    - Tiempo de instalación (tiempo total para instalar un producto)
    - Satisfacción del usuario (% de usuarios satisfechos con el producto)

• Vista de Productor - Mediciones

- Se concentra en **reducir las *pérdidas***
- Las mediciones se basan en
  - defectos y tasas de fallas
  - costos de re-trabajo (esfuerzo del personal)
- Las mediciones se aplican antes y después del ***release***

• Vista de Producto - Mediciones

Métricas *clásicas* de software

- Mediciones de complejidad de módulos
  - complejidad ciclomática de McCabe
- Mediciones de diseño
  - flujo de información

Problema

No está claro cómo estas mediciones afectan la calidad del producto

• Vista basada en Valor- Mediciones

- Costo y precio
- Fechas de entrega
- Productividad

• Costos y Beneficios de SQA

- Costos de SQA -- prevención y evaluación
- Beneficios de SQA -- asociados a disminución de ***rework*** (menos defectos y mayor confiabilidad)
- Importante que beneficios superen a los costos
- **Implicancia**: para ganar hay que invertir

ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

¡1 Cosa!
¿Qué recuerdas?

¿Qué rescato o creo que es importante recordar sobre la SQA?

DuocUC®

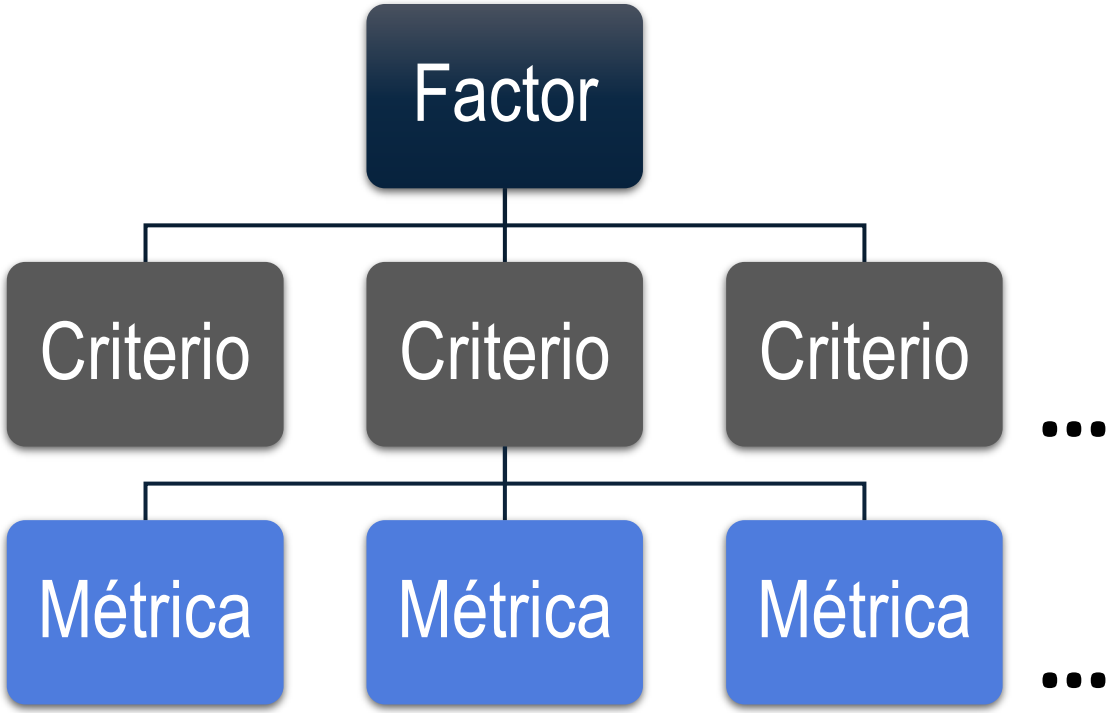
- **Micro Break**

02

Modelos de Calidad de Software

• Modelos de Calidad

- Modelos de calidad intentan **reconciliar diferentes visiones**
- Normalmente, identifican calidad de producto **importante para los usuarios**
- Ligadas a características y **atributos externos**

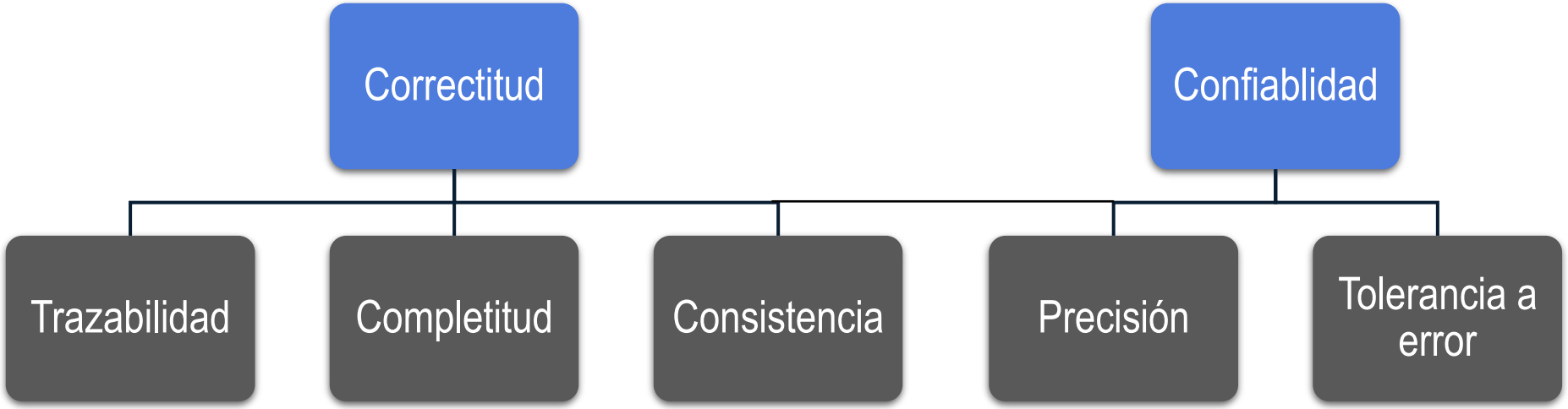


• Modelos de Calidad de McCall

- Modelo antiguo (1978), tiene 11 factores de calidad y 23 criterios de calidad

• Modelos de Calidad de McCall - Factores

- Un criterio puede afectar más de un factor



Problemas

- ¿cómo se sabe cuándo un modelo está completo?
- ¿cómo se decide qué criterios afectan qué factor?
- ¿cómo se decide qué es factor y qué es criterio?

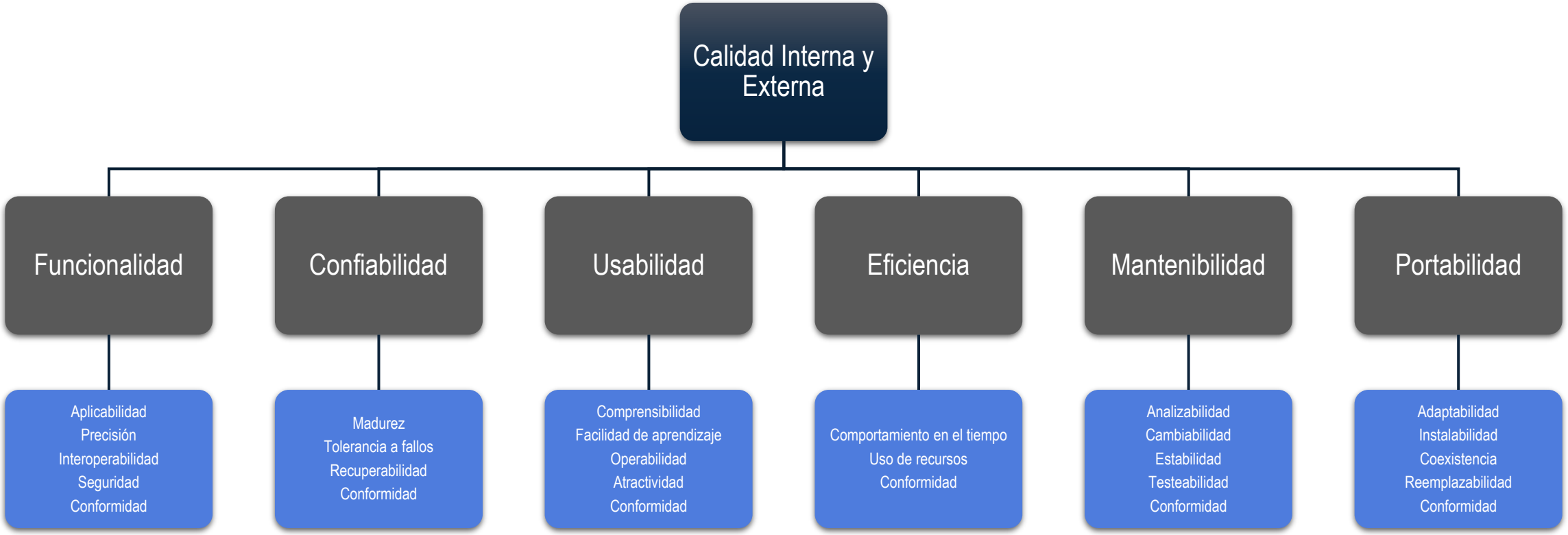
• Modelos de Calidad – Norma ISO/IEC 9126

- Estándar para medir calidad (1991)
- 6 características básicas, descompuestas en 20 subcaracterísticas (cada una se relaciona con solo una característica)

  Características de calidad: Un conjunto de atributos de un producto de software a través del cual su calidad es descrita y evaluada. Una característica de calidad de software puede ser refinada en múltiples niveles de subccaracterísticas.

- Refleja una visión de usuario, mientras el modelo de McCall refleja una visión de producto

• Modelos de Calidad – Norma ISO/IEC 9126



• Modelos de Calidad – Norma ISO/IEC 9126

Algunos problemas

- No es claro si el modelo es completo
  - no incluye **reusabilidad**
  - ***mantenibilidad*** abarca mejoras y corrección de defectos

- No es claro si algunas subcaracterísticas deben ser características
  - interoperabilidad v/s portabilidad

ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

¡1 Cosa!
¿Qué recuerdas?

¿Qué rescato o creo que es importante recordar sobre los modelos de calidad?

DuocUC®

- **Micro Break**

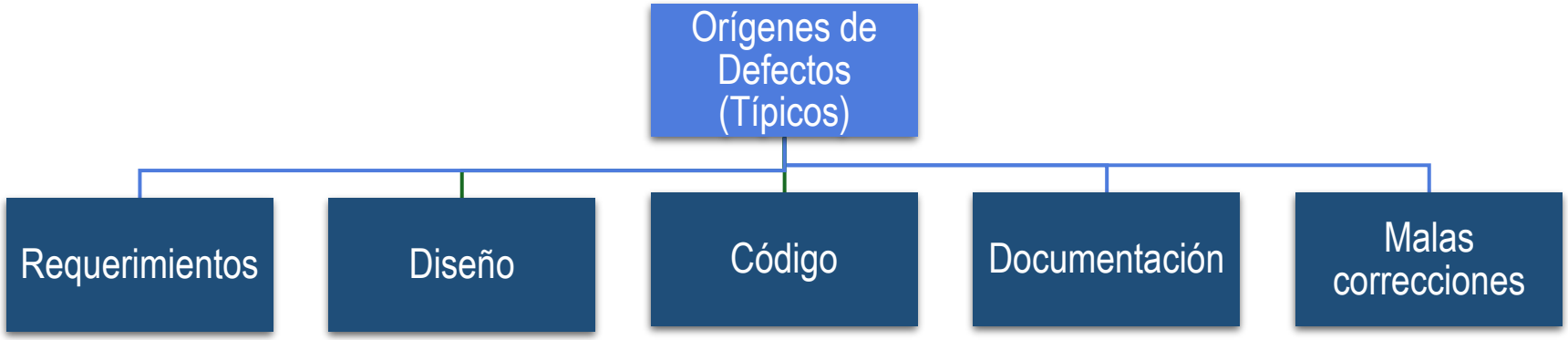
03

Defectos de Software

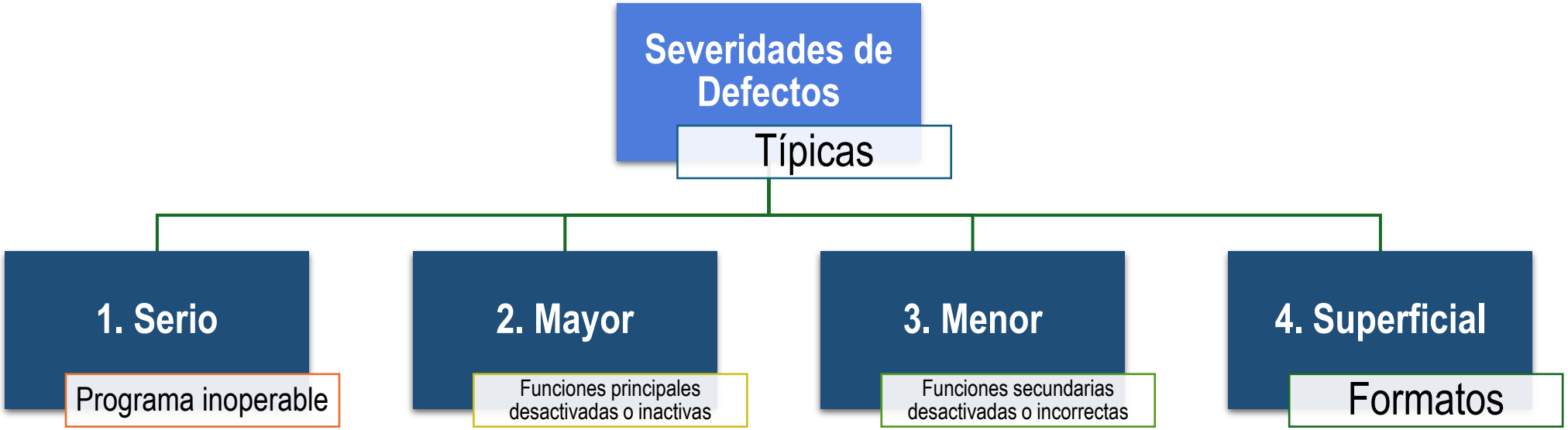
• Defectos de Software

- **Definición**: Problema o falla que causa que un programa o sistema falle completamente o entregue resultados incorrectos.

- No sirve "***no conformidad con los requerimientos***"



• Defectos de Software



• Defectos de Software – Remoción de defectos

- **Métrica fundamental**, debe recolectarse con frecuencia (típicamente mensual o trimestral)

- **Información recolectada**: defectos detectados, sus severidades y orígenes en las diversas etapas de la remoción de defectos (*prerelease*)

  - revisión de requerimientos
  - revisión de diseño
  - inspección de código
  - pruebas de integración
  - pruebas de aceptación

- **Idea**: **comenzar a contabilizar lo antes posibles**

• Defectos de Software – Distribución de defectos

Distribución de Orígenes y Severidades de Defectos de Software

	Nivel de Severidad				
Origen de Defecto	*1, %*	*2, %*	*3, %*	*4, %*	*Total , %*
Requerimientos	5,0	5,0	3,0	2,0	15,0
Diseño	3,0	22,0	10,0	5,0	40,0
Codificación	2,0	10,0	10,0	8,0	30,0
Documentación	0,0	1,0	2,0	2,0	5,0
Malas Correcciones	0,0	2,0	5,0	3,0	10,0
Total de Defectos	10,0	40,0	30,0	20,0	100,0

• Defectos de Software – Distribución de defectos

Distribución de Defectos de Software por Pasos de Remoción

Actividad de Remoción	Defectos Encontrados	Defectos por KLOC	Defectos por Puntos de Función
Revisión de Requerimientos	125	2,5	0,25
Revisión de Diseño	250	5,0	0,50
Inspección de Código	500	10,0	1,00
Test de Integración	150	3,0	0,30
Test de Aceptación	100	2,0	0,20
Total de Defectos	1125	22,5	2,25

• Defectos de Software – Distribución de defectos

Defectos Informados por el Usuario en 1 Año de Uso del Producto

Actividad	*Defectos Encontrados*	*Defectos por KLOC*	*Defectos por Punto de Función*
Defectos reportados por el usuario el primer año	400	8,0	0,8

- Recolección de datos durante 1er, 2do, etc. año de operación
- **Eficiencia en la remoción**: % de defectos recolectados antes de la entrega (*prerelease*)
- Importancia de aumentar la **eficiencia en la remoción** pues impacta la **calidad**, **productividad** y la **satisfacción del usuario**

- **Defectos de Software – Caso Hewlett Packard**

- **Eficiencia** (defectos detectados por hora)
  - Test regular ..... (0.21)
  - Caja negra ..... (0.282)
  - Caja blanca ..... (0.322)
  - Inspecciones ..... (1.057)

- **Costos por Defectos**

La Paradoja de Costo por Defecto Para Medición de Calidad

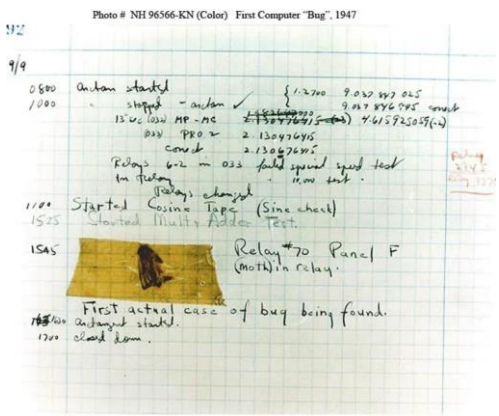
	Aplicación Ada de Baja Calidad	Aplicación Ada de Alta Calidad
Tamaño en KLOC	10	10
Tamaño en Puntos de Función	150	150
Defectos Encontrados	500	5
Costos de Preparación	$ 5,000	$ 5,000
Costos de Ejecución	$ 5,000	$ 2,500
Defectos Reparados	$ 25,000	$ 2,500
Costo Total de Remoción	$ 35,000	$ 10,000
Costos por Defectos	$ 70	$ 2,000
Costos por Puntos de Función	$ 233,33	$ 66,66

ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

¡1 Cosa!
¿Qué recuerdas?

¿Qué rescato o creo que es importante recordar sobre los defectos del software?

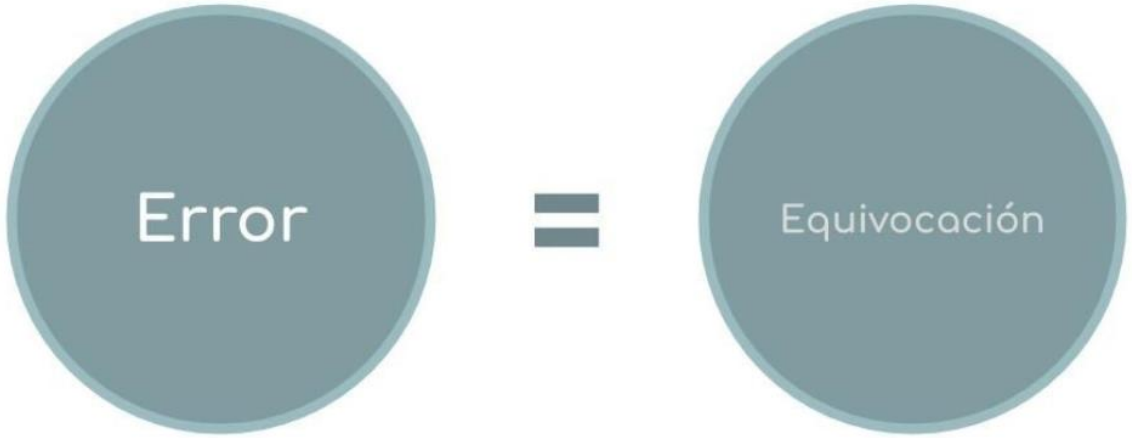
DuocUC®



• ERROR DEFECTO Y FALLO

¿Cuál es la diferencia entre los tres?

Un error es lo mismo que una equivocación, que por lo general es cometida por una persona, ya sea en el código del software o en algún otro producto de trabajo.



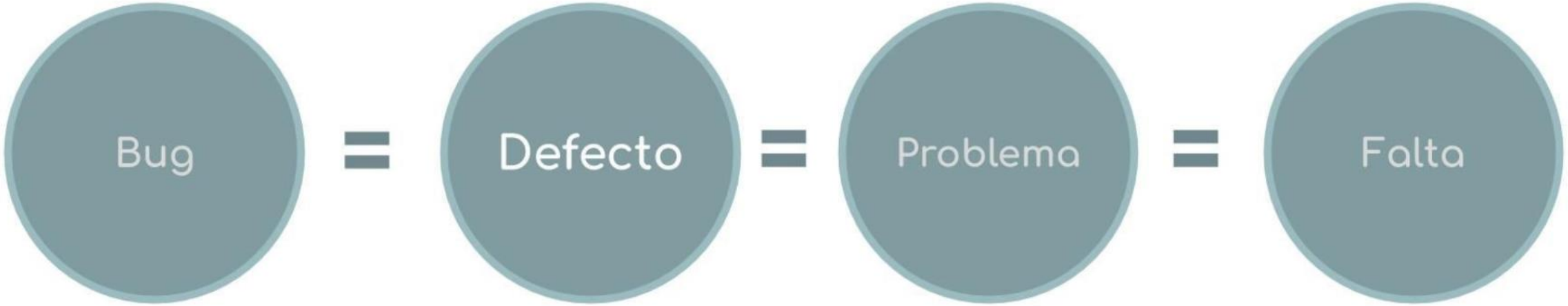
Cuando se comete un error, este introduce un defecto.

Un defecto, es lo mismo que un problema o falta y también es conocido como ***bug*** por los desarrolladores.

• ERROR DEFECTO Y FALLO

¿Cuál es la diferencia entre los tres?

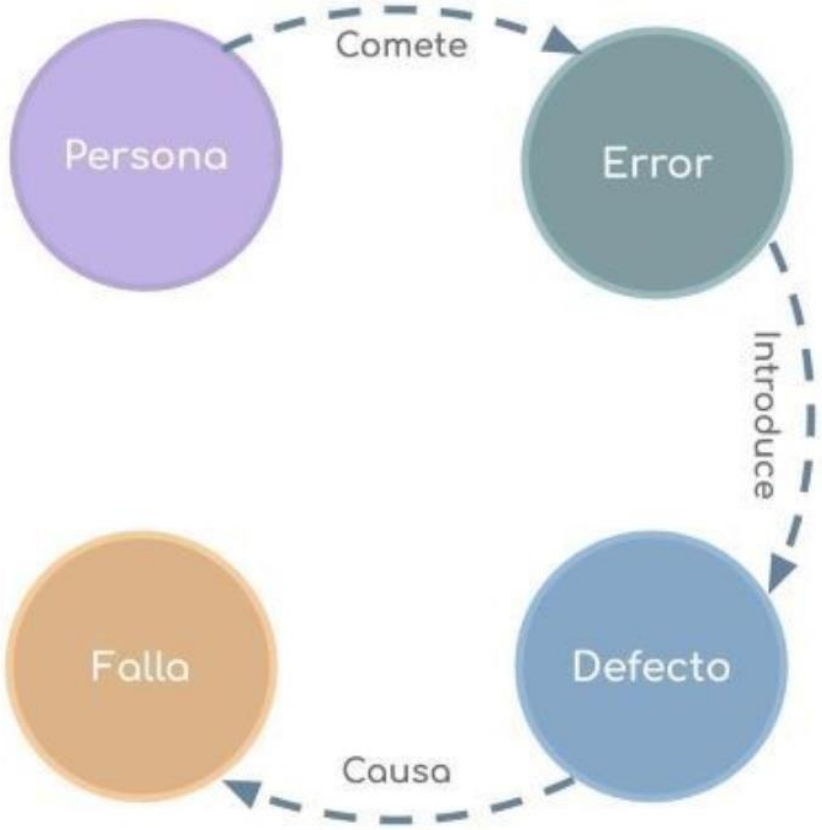
Un defecto puede definirse como una imperfección en un componente o sistema que puede causar que este no desempeñe las funciones requeridas.



• ERROR DEFECTO Y FALLO

¿Cuál es la diferencia entre los tres?

Si un fragmento de código que contiene un defecto se ejecuta, es posible que cause un fallo, aunque esto no siempre pasa en todas las circunstancias, a veces se requiere de situaciones y datos muy específicos para que ocurra un fallo.



• ERROR DEFECTO Y FALLO

¿Ejemplo de la diferencia entre los tres:

Imagine que se le ha pedido al desarrollador que construya un sitio web en el cual solo pueden acceder personas de 15 años o más y él desarrolla un algoritmo parecido al siguiente:

```
if (edad > 15) {
  alert("Bienvenido a nuestro sitio web");
} else {
  alert("No puede acceder a este sitio");
}
```

La especificación incluye la edad de 15 años como una edad permitida y el algoritmo la excluye, por lo que el desarrollador cometió un error e introdujo un defecto en la condición.

• ERROR DEFECTO Y FALLO

Causando el siguiente resultado en los casos de prueba:

AÑOS	RESULTADO ESPERADO	RESULTADO ACTUAL
14	No puede acceder a este sitio	No puede acceder a este sitio
15	Bienvenido a nuestro sitio web	No puede acceder a este sitio
16	Bienvenido a nuestro sitio web	Bienvenido a nuestro sitio web

Con el defecto en el código, al introducir la edad 15 en la plataforma, aparecerá el siguiente mensaje:

No puede acceder a este sitio

Este es el fallo ocasionado por el defecto del código que fue causado por el error cometido por el desarrollador.

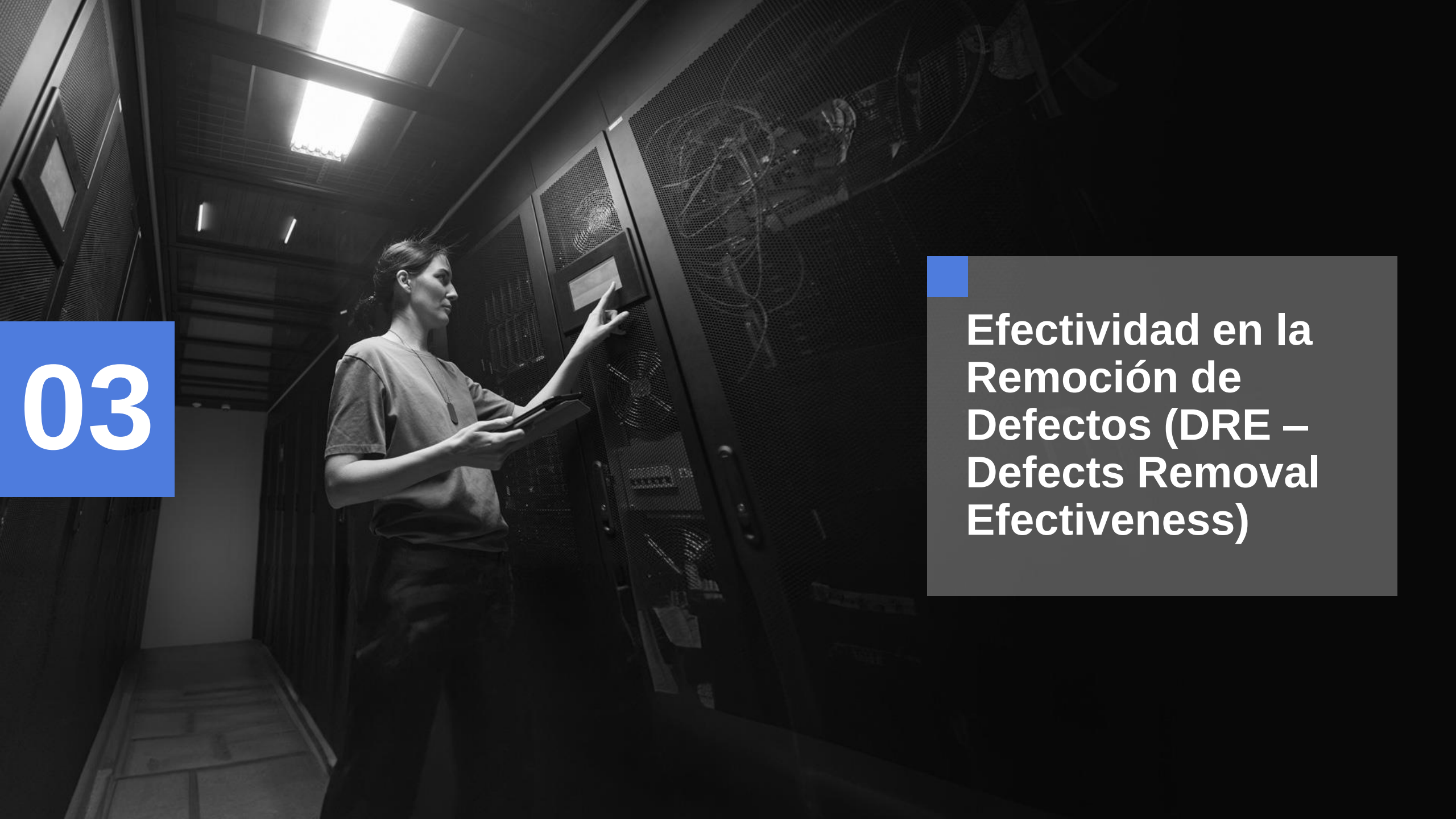
ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

¡1 Cosa!
¿Qué recuerdas?

¿Qué rescato o creo que es importante recordar sobre los errores, defectos y fallos del software?

DuocUC®

- **Micro Break**



03

Efectividad en la Remoción de Defectos (DRE – Defects Removal Efectiveness)

• Efectividad en la Remoción de Defectos

- Para definir claramente la **efectividad de la eliminación de defectos**, primero debemos comprender las actividades en el proceso de desarrollo que están relacionadas con las **inyecciones** y **eliminaciones de defectos (detección)**.

- Los defectos se inyectan en el producto o en los entregables intermedios del producto (p. Ej., Documento de diseño) y en varias fases.

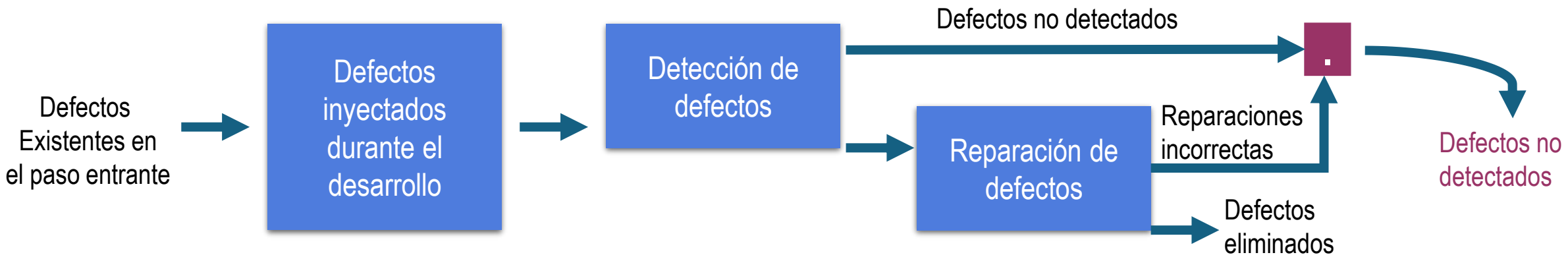
Es incorrecto suponer que todos los defectos del software se inyectan al comienzo del desarrollo

• Efectividad en la Remoción de Defectos

- La siguiente tabla muestra un ejemplo de las actividades en las que se pueden inyectar o eliminar defectos para un proceso de desarrollo.

Fase Desarrollo	Inyección de Defectos	Eliminación de Defecto
Requisitos	Proceso de recopilación de requisitos y desarrollo de especificaciones funcionales de programación.	Análisis de requisitos y revisiones
Diseño de Alto Nivel	Tareas de Diseño	Inspecciones de diseño de alto nivel
Diseño de Bajo Nivel	Tareas de Diseño	Inspecciones de diseño de bajo nivel
Codificación	Programación	Inspección de código
Integración	Integración de sistema	Verificación y pruebas de integración
Testing Unitario	Malas correcciones	Testing
Testing Componentes	Malas correcciones	Testing
Test de Sistema	Malas correcciones	Testing

- **Inyección y eliminación de defectos durante un paso del proceso**



$$DRE = \frac{Defectos\ Removidos\ en\ la\ Etapa}{Defectos\ Existentes\ en\ la\ Etapa(Escapados\ de\ la\ etapa\ anterior) + Defectos\ inyectados\ durante\ el\ desarollo\ de\ la\ etapa}$$

• Efectividad en la Remoción de Defectos

- Para derivar una definición operativa, se utiliza un enfoque matricial mediante la clasificación cruzada de los datos de defectos en términos de la **fase de desarrollo en la que se encuentran (y eliminan) los defectos** y **las fases en las que se inyectan los defectos.**

- Esto requiere que para cada defecto encontrado, **su origen** (la fase en la que se introdujo) sea decidido por el grupo de inspección (para los defectos de inspección) o por acuerdo entre el tester y el desarrollador (para los defectos de prueba).

• Efectividad en la Remoción de Defectos

Donde se encontraron (removidos)	Origen del Defecto (Inyectados)								
	Requisitos (REQ)	Diseño de Alto Nivel (DA)	Diseño Detallado (DD)	Codificación (CO)	Testing Unitario (TU)	Test de Componentes (TC)	Test de Sistema (TS)	Producción (PR)	Total
Requisitos (REQ)	----								---
Diseño de Alto Nivel (DA)	49	681							730
Diseño Detallado (DD)	6	42	681						729
Codificación (CO)	12	28	114	941					1095
Testing Unitario (TU)	21	43	43	223	2				332
Test de Componentes (TC)	20	41	61	261	----	4			387
Test de Sistema (TS)	6	8	24	72	----	----	1		111
Producción (PR)	8	16	16	40	----	----	----	1	81
Total	122	859	939	1537	2	4	1	1	3465

Defect Removal Efectiveness (DRE)		

74%	Inspection Efectiveness	73,7%
61%		
55%		
37%	TEST Removal Efectiveness	91,1%
67%		
58%		
100%		
97,7%	Overall Defect Removal Efectiveness (DRE)	

$$\frac{Defectos\ Eliminados\ por\ Inspecciones}{Todos\ los\ defectos} \text{ x } 100\%$$

$$= \frac{730+729+1095}{3465} \text{ x } 100\%$$

$$= \frac{332+387+111}{332+387+111+81} \text{ x } 100\%$$

$$DRE = \left(1 - \frac{81}{3465}\right) x\ 100\%$$

• Costo de Efectividad en la Remoción de Defectos

- Además de la eficacia de eliminación de defectos por fase per se, el costo de eliminación de defectos debe tenerse en cuenta para una **planificación de calidad eficiente**.

- La eliminación de defectos en las primeras fases de desarrollo es generalmente **menos costosa**.

- Cuanto más cerca se encuentran los defectos en relación con el lugar y el momento en que se inyectan, **menor es el esfuerzo de remoción y reelaboración**.

ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

¡1 Cosa!
¿Qué recuerdas?

¿Qué rescato o creo que es importante recordar sobre la efectividad de la remoción de defectos?

DuocUC®

• Resumen

- La eliminación efectiva de defectos durante el proceso de desarrollo es fundamental para **el éxito de un proyecto de software**.

- La importancia del concepto de eficacia en la eliminación de defectos y su medición es bien reconocida. La literatura y los ejemplos de la industria corroboran la hipótesis de que la eliminación eficaz de los defectos iniciales conduce a **una mejor calidad del producto final**.

- El costo relativo de la eliminación de defectos en el front-end es mucho **menor que el costo** de las pruebas formales en el back-end y la fase de mantenimiento cuando el producto está en producción.

• Resumen

- Para medir la eficacia de eliminación de defectos de fase, es mejor utilizar el **enfoque de matriz** en el que los datos de defectos se tabulan en forma cruzada en términos de origen del defecto y la fase en la que se encuentran los defectos.

- En general, cuanto más corto sea el tiempo entre el origen del defecto y el descubrimiento del defecto, **más efectivo y menos costoso** será el proceso de desarrollo.

- Se ha evaluado la eficacia de la eliminación de defectos por el **nivel de madurez del proceso** y se han establecido líneas de base de comparación

ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

Demuestra lo que aprendiste

socrative by MasteryConnect

DuocUC®

• Bibliografía

- Pressman, R. S., & Maxim, B. R. (2021). Modelos de procesos de software. En Ingeniería de software: Un enfoque práctico (9na ed., pp. 50-74). McGraw-Hill Education.

- International Organization for Standardization. (2023). Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models (ISO/IEC 25010:2023). https://www.iso.org/standard/78176.html

DuocUC®

CERCANÍA. LIDERAZGO. FUTURO.

duoc.cl

7 AÑOS ACREDITADO

Comisión Nacional de Acreditación CNA-Chile

DESDE AGOSTO 2017 HASTA AGOSTO 2024.
DOCENCIA DE PREGRADO. GESTIÓN INSTITUCIONAL. VINCULACIÓN CON EL MEDIO.